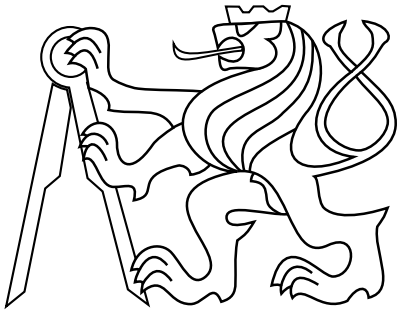
Czech Technical University in Prague

Faculty of Electrical Engineering
Department of Cybernetics
Multi-robot Systems



Learning an Autonomous Drone Racing Policy via Differentiable Simulation

Master's Thesis

Adam El Ghaib

Prague, January 2026

Study programme: Cybernetics and Robotics (Space Master)

Supervisor: Ing. Robert Pěnička, Ph.D.

Abstract

Todo

Keywords Unmanned Aerial Vehicles, Autonomous Drone Racing, Simulation to Reality Transfer, Reinforcement Learning, Domain Randomization

Abbreviations

API Application Programming Interface

DOFs Degrees-Of-Freedom

LiPo Lithium Polymer

IMU Inertial Measurement Unit

VIO Visual Inertial Odometry

EKF Extended Kalman Filter

MPC Model Predictive Control

DFBC Differential Flatness Based Control

UAV Unmanned Aerial Vehicle

RL Reinforcement Learning

DR Domain Randomization

POMDP Partially Observable Markov Decision Process

DoF Degrees of Freedom

Sim2Real Simulation-to-Reality

PPO Proximal Policy Optimization

TRPO Trust Region Policy Optimization

NN Neural Network

MLP Multi-Layer Perceptron

MLPs Multi-Layer Perceptrons

CNNs Convolutional Neural Networks

CRNN Convolutional Recurrent Neural Network

CTBR Collective Thrust and Body Rates

ADR Autonomous Drone Racing

ADE Automatic Differentiation Engine

OCP Optimal Control Problem

OCPs Optimal Control Problems

SRT Single Rotor Thrust

CTBR Collective Thrust and Body Rates

LVHR Linear Velocities and Heading Rate

TWR Thrust to Weight Ratio

ML Machine Learning

BPTT Backpropagation Through Time

SGD Stochastic Gradient Descent

RMSProp Root Mean Square Propagation

DC Direct Current

ESC Electronic Speed Controller

RMSE Root Mean Square Error

DP Dynamic Programming

DQN Deep Q-Learning

SR Success Rate

Contents

1 Introduction **1**
1.1 Background . . . 1
1.2 Related Work . . . 3
1.3 Objectives and Structure . . . 4

2 Preliminaries **6**
2.1 Optimal Control Problem . . . 6
2.1.1 General Discrete-Time Optimal Control Problem . . . 6
2.1.2 Minimum-Time Optimal Control Problem . . . 7
2.1.3 Simplified Minimum-Time Optimal Control Problem . . . 7
2.2 Classical Control approaches . . . 8
2.2.1 Model Predictive Control . . . 8
2.2.2 Differential Flatness Based Control . . . 9
2.3 Learning-Based approaches . . . 10
2.3.1 Model-Free Reinforcement Learning . . . 10
2.3.2 Model-Based Reinforcement Learning . . . 11
2.4 Differentiable Optimization . . . 12
2.4.1 Computational Graph and Backpropagation . . . 12
2.4.2 Optimization Process . . . 14

3 Methodology **16**
3.1 Quadrotor Model . . . 16
3.2 Control Abstraction . . . 18
3.2.1 Single Rotor Thrust . . . 18
3.2.2 Collective Thrust and Body Rates . . . 19
3.2.3 Linear Velocities and Heading Rate . . . 19
3.2.4 Linear Velocity Heading Rate and Gains . . . 21
3.3 Domain Randomization . . . 21
3.4 Training Task . . . 23
3.4.1 Trajectory Tracking . . . 23
3.5 Summary . . . 25

4 Results **28**
4.1 Evaluation Criteria . . . 28
4.1.1 Sample Efficiency . . . 28
4.1.2 Perfromance . . . 29
4.1.3 Transferability and Robustness . . . 30
4.2 Control Abstraction Evaluation . . . 30
4.2.1 Sample Efficency . . . 30

4.2.2 Performance . . . 31
4.2.3 Transferability . . . 35
4.3 Surrogate Gradient Analysis . . . 35
4.3.1 Robustness . . . 36
4.4 Real World Validation . . . 36
4.5 Model-Free and Model-Based evaluation . . . 36
4.5.1 Training Time . . . 36
4.5.2 Discrete and Continuous Rewards . . . 37
4.5.3 Performance . . . 37
4.5.4 Transferability and Robustness . . . 38

5 Conclusion **39**

6 Future Directions **40**

7 References **41**

A Appendix A **45**
A.1 Simulator Validation . . . 45

Chapter 1

Introduction

In this chapter we brifely introduce the reader to the sport of Autonomous Drone Racing (ADR), its associated challeneges and the current state of the art to adress them. Finally, we provide an overvirew of the structure and objectives of this thesis.

1.1 Background

ADR has emerged as a compelling platform in robotics to develop and deploy novel control techniques in real-world scenarios. The aim of this sport is to navigate through a sequence of gates in minimum time, pushing the platform's capabilities to their limits [1]. To maintain agility and low weight, the hardware is kept minimal; a typical ADR drone consists of a quadrotor equipped with an RGB camera, an Inertial Measurement Unit (IMU), an onboard computer, a flight controller, brushless Direct Current (DC) motors with propellers, and an Electronic Speed Controller (ESC) all powered by a Lithium Polymer (LiPo) battery [2, 3].

ADR presents several challenges, which can be divided into the following subgroups depicted in Figure 1.1.

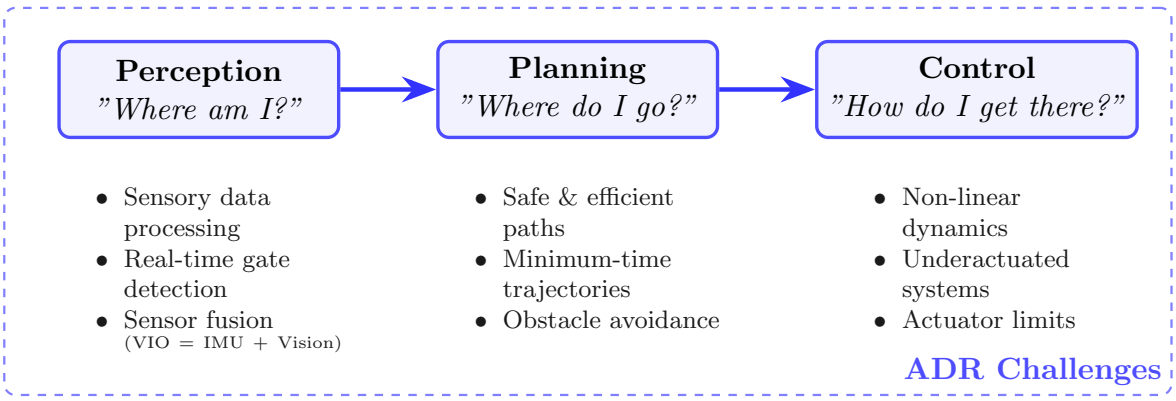


Figure 1.1: The modular navigation stack for ADR and its associated challenges.

The control stack begins at the **perception** level. This stage involves processing sensory data from the camera and IMU, which is inherently noisy and uncertain. The main challenge at this level is to filter and fuse this data to estimate both the position of the gates and the

state of the drone. The fusion of camera and IMU measurements is known as Visual Inertial Odometry (VIO). State estimation is typically performed using filtering techniques that combine a system model with sensor measurements, such as a Kalman Filter [4] or variants [5]. Given an estimate of the drone state and the environment, the next step is **planning**. The objective in ADR is to compute a minimum-time trajectory that passes through the gates while avoiding collisions. Trajectory planning can be computationally demanding, making efficient approaches particularly important. The final stage of the pipeline is **control**. Given a planned trajectory, the goal is to determine the control inputs that track it with minimal error. This task is challenging due to the highly nonlinear dynamics of the quadrotor. Additionally, the system is underactuated, with only four out of the six Degrees-Of-Freedom (DOFs) directly controllable. Moreover, yaw control authority is typically limited, and actuator saturation further constrains performance.

Many of these challenges are common in robotics, and a variety of methods have been proposed to address them. Learning-based methods, particularly those relying on Neural Network (NN), are gaining popularity due to their low-latency inference and robustness to noise. Individual components of the navigation stack can be learned using NNs. For instance, Convolutional Neural Networks (CNNs) can process VIO data to detect gates, while Multi-Layer Perceptrons (MLPs) can be used for planning or control. Combinations of multiple stages, such as perception and planning or planning and control, can also be learned jointly. In some cases, the entire pipeline is learned end-to-end [1]. Learning-based approaches offer several advantages; however, training these models typically requires large amounts of data, and current optimization methods can be sample inefficient. Since collecting such data in real-world settings is often impractical, simulated environments that approximate real-world distributions are commonly used, enabling thousands of parallel training instances. Recent research has also explored the use of Automatic Differentiation Engine (ADE) in Machine Learning (ML) frameworks [6, 7, 8] to propagate gradients from a cost function to policy parameters. This approach significantly improves sample efficiency [9] and avoids treating the quadrotor dynamics as a black box.

In this thesis, we investigate this direction by focusing on the use of differentiability to learn the control component of the navigation stack. We aim to demonstrate that this approach is a promising avenue for research in robotics and particulalry in ADR.

Figure 1.2: Long eexposure photography in the a2rl drone racing championship.

1.2 Related Work

The evolution of ADR has progressed from early academic benchmarks in the IROS competitions between 2016 and 2018 [10, 11, 12], where drones reached speeds of only a few meters per second, to highly professionalized racing events since 2019, where autonomous drones can exceed 150 km/h and outperform professional human pilots.

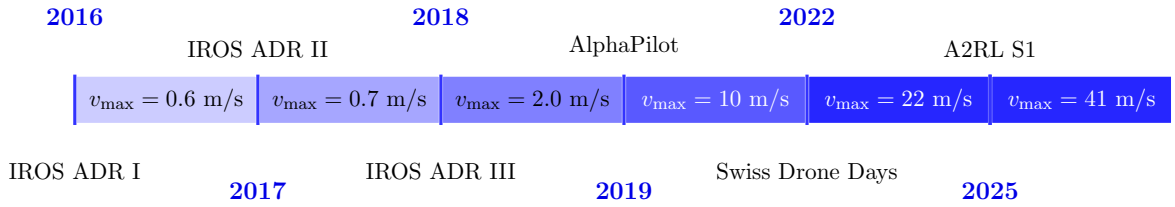


Figure 1.3: Maximum reached speeds over time (adapted from [1]).

Since the AlphaPilot competition [13], there has been a clear trend toward incorporating learning-based components into the navigation stack. In [14], the pipeline combined VIO and gate detections within an Extended Kalman Filter (EKF) for state estimation, followed by trajectory planning using sampling-based receding horizon methods. Control was implemented through a cascade of position and attitude controllers, while NNs were used exclusively for visual gate detection. Subsequent work in [15] retained a similar perception pipeline but replaced planning and control with a feedforward Multi-Layer Perceptron (MLP), optimized using Proximal Policy Optimization (PPO) [16].

In a related setup, [17] demonstrated that model-free Reinforcement Learning (RL) could outperform traditional optimal control approaches. The authors attribute the increase in performance of RL due to two main factors. First, the ability of model-free RL to directly formulate the task goal as part of the optimization function without the need of decomposing the the problem into planning and control. This allows their policy to explore a broader range of behaviours thath provide robustness to unmodeled dynamics effects. Furthermore, since RL is usually modeled as a Markov Decision Process, the tranisition model is probabilisitc, which allows for Domain Randomization (DR). "By training in randomized environments, the agent becomes more robust and adaptable, making it more effective at handling disturbances and model uncertainty" [17].

On a similar note, recent approaches have explored model-based RL, where a learned world model enables training through imagined rollouts. In [18, 19], the authors adopt an actor-critic architecture inspired by [20]. This approach achieved champion-level performance and has the advantage of directly mapping from visual data, raw pixel information, to direct motor commands skipping the need of implementing an intermediate controller. While these methods achieve strong performance, they suffer from high training times and limited sample efficiency; for instance, training in [19] can take up to 50 hours, significantly slowing down the research cycle.

To address these limitations, recent work has shifted toward differentiable approaches that enable backpropagation of gradients from an objective function through the entire navigation stack, this is known as Backpropagation Through Time (BPTT). This paradigm avoids treating the quadrotor dynamics as a black box and substantially improves sample efficiency. The work of [9] pioneered this direction, achieving agile vision-based flight in cluttered environments using a Convolutional Recurrent Neural Network (CRNN) and a simplified double-

integrator model. A differentiable simulation framework is introduced in [21], which has inspired the simulator design used in this work. Similarly, [22] presents an efficient differentiable quadrotor framework implemented in JAX for learning stabilization from visual features, and compares it against standard PPO. Building on this line of work, [23, 24] demonstrate real-time policy adaptation using real-world data, leveraging the improved sample efficiency of differentiable methods.

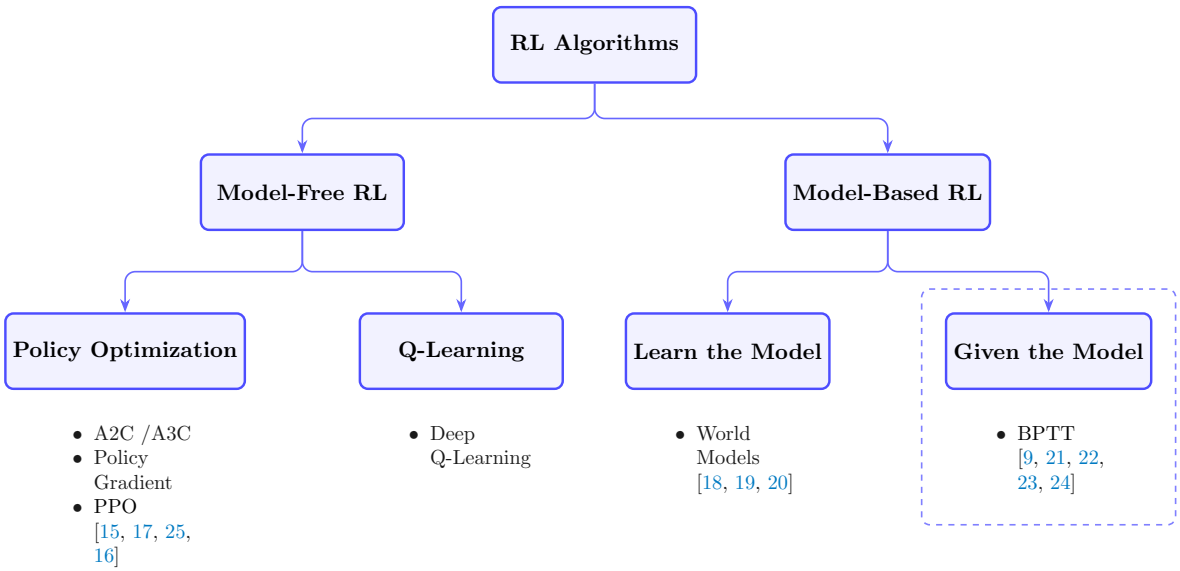


Figure 1.4: The modular navigation stack for ADR and its associated challenges.

On the other hand, another research direction relevant for this thesis, aims to answer the question of how to tune the gains of a given controller to achieve optimal performance. The authors of [26] show that the faster a trajectory to be tracked is, the more sensitive is the tracking performance to the controller gains. This way, they propose an algorithm based on the Metropolis-Hastings sampling [27], to tune the gains of a Model Predictive Control (MPC) controller for different segements of a trajectory. The trajectory is divded in segments with similar height gradients and minimum time of 2 s per segment. Their approach showed an increase on trajectory tracking succes comparted to another automatic gain tuning method absed on genetic algorithms [28].

1.3 Objectives and Structure

The primary objective of this thesis is to create a differentiable quadrotor simulator to backpropagate the gradients from a given cost function to the NN parameters to command autonomusly a drone. A low-level geometric controller will take the velocity and heading rate commands from the newtork and map them to wrench commands. Aditionally, the policy will be agumented to output the gains of the low-level geometric $SO(3)$ controller.

As secondary objectives, we perfrom an evaluation of the policy network on Gazebo uttilizing the MRS UAV stack [29], and we test the controller on a real platform. Furthermore, we perform a comparative study between different control modalities, and, model-free against model-based optimization methods.

Table 1.1: Thesis Objectives

	Primary Objectives
Obj-1.1	Create a differentiable quadrotor simulator.
Obj-1.2	Train a NN that outputs velocity and heading commands.
Obj-1.3	Augment the NN to output adaptive gains for the $SO(3)$ controller.
	Secondary Objectives
Obj-2.1	Evaluate how other control modalities compare to geometric control.
Obj-2.2	Compare model-based optimization against model-free.

The thesis is divided in 6 chapters. In Chapter 1 provide and introduction of the thesis topic and cover the most relevant related work. In Chapter 2, we perf In Chapter 3, we present the methodology of our method, we explain the learning environment, the quadorotr model, the control modalities thath have been tested, the domain randomization strategy and we formulate the cost function to be minimized. In Chapter 4, we show the results of our method. We analyze the effect of the different control abstarctions on the sample efficency and performance. Additionally we show the effect of other implementation decisitons and we analyze real world flight data. In Chapter 5, we discuss the results of our method. Finally, in Chapter 6 we give future research directions.

Chapter 2

Preliminaries

This chapter covers relevant theory for this thesis.

TODO

2.1 Optimal Control Problem

According to [30], optimal control was born in 1687 in the Netherlands when Johan Bernoulli published his solution to the brachystochrone problem. Optimal control can be regarded as an extension of calculus of variations, with the aim of finidng a control policy that minimizes a cost function. In this section, we formulate optimal control problems related to this thesis.

2.1.1 General Discrete-Time Optimal Control Problem

In general, a nonlinear discrete-time Optimal Control Problem (OCP) can be formulated as the optimization of a running cost (or loss) function $L_k()$, plus a terminal cost function $\phi()$, subject to the system's dynamics $\mathbf{f}_k()$, and the set of permitted states $\mathcal{X}_k$, and controls $\mathcal{U}_k$.

$$\begin{aligned} \underset{\mathbf{X},\,\mathbf{U}}{\text{minimize}} \quad & \left(\phi(\boldsymbol{x}_N, N) + \sum_{k=0}^{N-1} L_k(\boldsymbol{x}_k, \boldsymbol{u}_k) \right) \\ \text{subject to} \quad & \boldsymbol{x}_{k+1} = \mathbf{f}_k(\boldsymbol{x}_k, \boldsymbol{u}_k), \quad k = 0, \ldots, N-1, \\ & \boldsymbol{u}_k \in \mathcal{U}_k, \quad k = 0, \ldots, N-1, \\ & \boldsymbol{x}_k \in \mathcal{X}_k, \quad k = 0, \ldots, N, \end{aligned} \tag{2.1}$$

Where

N is the final discrete time,
$\boldsymbol{x}_k \in \mathbb{R}^n$ is the state at the discrete time $k \in \mathbb{Z}$,
$\boldsymbol{u}_k \in \mathbb{R}^m$ is the control at the discrete time $k \in \mathbb{Z}$,
$\mathbf{X} = [\boldsymbol{x}_0, \ldots, \boldsymbol{x}_N]$ is the trajectory of states,
$\mathbf{U} = [\boldsymbol{u}_0, \ldots, \boldsymbol{u}_{N-1}]$ is the trajectory of control actions.

2.1.2 Minimum-Time Optimal Control Problem

The last formulation of an OCP can be further specified for the case of drone racing in several manners. Since the main objective is to complete a lap through the racing track as fast as possible, this problem can be written as minimum-time problem.

$$\begin{aligned} \underset{\mathbf{U},\, N}{\text{minimize}} \quad & \sum_{k=0}^{N-1} 1 \\ \text{subject to} \quad & \boldsymbol{x}_{k+1} = \mathbf{f}_k(\boldsymbol{x}_k, \boldsymbol{u}_k), \quad k = 0, \ldots, N-1, \\ & \boldsymbol{u}_k \in \mathcal{U}_k, \quad k = 0, \ldots, N-1, \\ & \boldsymbol{x}_k \in \mathcal{X}_k, \quad k = 0, \ldots, N, \end{aligned} \tag{2.2}$$

The racing track restrictions, i.e. passing through a sequence of gates while avoiding obstacles, can be introduced as hard constraints under the set of permitted states $\mathcal{X}_k$ as

$$\mathcal{X}_k = \left\{ \boldsymbol{x}_k \in \mathbb{R}^n \;\middle|\; \boldsymbol{p}_k \in \mathcal{G}_{g(k)}, \quad \boldsymbol{p}_k \notin \mathcal{O}_j, \quad \forall\, j \in \{1, \ldots, M\} \right\}, \tag{2.3}$$

where

$\boldsymbol{p}_k \in \mathbb{R}^3$ is the position of the drone at time step k,
$\mathcal{G}_{g(k)}$ is the feasible region defined by gate g at time step k, with $g(k) \in \{1, \ldots, G\}$ denoting the index of the gate to be passed at k,
G is the total number of gates in the racing track,
$\mathcal{O}_j$ is the region occupied by the j-th obstacle,
M is the total number of obstacles.

2.1.3 Simplified Minimum-Time Optimal Control Problem

The original problem formulation (subsection 2.1.2) is inherently complex, as it requires simultaneously solving for an optimal trajectory that clears all gates and avoids obstacles while determining the optimal control sequence achievable within the vehicle's dynamics and actuator constraints. To make the problem more tractable, this thesis assumes that a reference trajectory or path is provided a priori. Consequently, the OCPs addressed hereafter focus on trajectory tracking or path progress, formulated as follows:

Trajectory Tracking

In the trajectory tracking OCP, the objective is to find the optimal control sequence $\mathbf{U}$ that minimizes the error between a given reference state $\boldsymbol{x}_k^{\text{ref}}$ and the current system state $\boldsymbol{x}_k$, subject to dynamics, actuator, and state constraints. While this simplifies the optimization significantly it comes at the cost of reduced robustness. By assuming the trajectory is fixed, the system cannot re-optimize the path to accommodate state constraints or environmental changes, effectively limiting the drone's agility compared to a unified spatial-temporal optimization.

$$\begin{aligned} \underset{\mathbf{U}}{\text{minimize}} \quad & \sum_{k=0}^{N-1} \left\| \boldsymbol{x}_k - \boldsymbol{x}_k^{\text{ref}} \right\|_{\mathbf{Q}}^2 \\ \text{subject to} \quad & \boldsymbol{x}_{k+1} = \mathbf{f}_k(\boldsymbol{x}_k, \boldsymbol{u}_k), \quad k = 0, \ldots, N-1, \\ & \boldsymbol{u}_k \in \mathcal{U}_k, \quad k = 0, \ldots, N-1, \\ & \boldsymbol{x}_k \in \mathcal{X}_k, \quad k = 0, \ldots, N, \end{aligned} \tag{2.4}$$

2.2 Classical Control approaches

Given the OCP formulated in last sections, by solving the quadratic program, a trajectory of control inputs that minimizes the cost function can be obtained. However, in this direct approach, the control is open-loop. Henece it is not robust to external disturbances and model mistmaches. In the classical literature, a well stablished approach to adress this issue is MPC.

On the other hand, "the states and the inputs can be written as algebraic functions of four carefully selected flat outputs and their derivatives" [31]. Leveraging this property, motors command can be directly obtained from a given trajectory to be tracked.

Many other quadrotor control methods are well stablished on the lituerature, ranging from Proportional Derivative Integral (PID) cotnrollers, Linear Quadratic Regulators (LQR), $\mathcal{L}_2$ adaptive control, H_∞ norm minimization, μ-sythesis and many others, each with their own advatages and disatvantages. Howver, in this section we give an overview of MPC and Differential Flatness Based Control (DFBC) as we find them the most relevant for this thesis.

2.2.1 Model Predictive Control

Reciding horizion control or also known as MPC is a direct approach to solving an OCP. The key idea behind this method is to solve the control problem for a given prediction horizion of N_p steps. As a solution we obtain the control inputs that minimize the cost function for that horizion, nonetheless we only take 2 or 3 steps, N_c of those controls and discard the rest. Then we solve again the control problem with the updated states, and repeat the process.

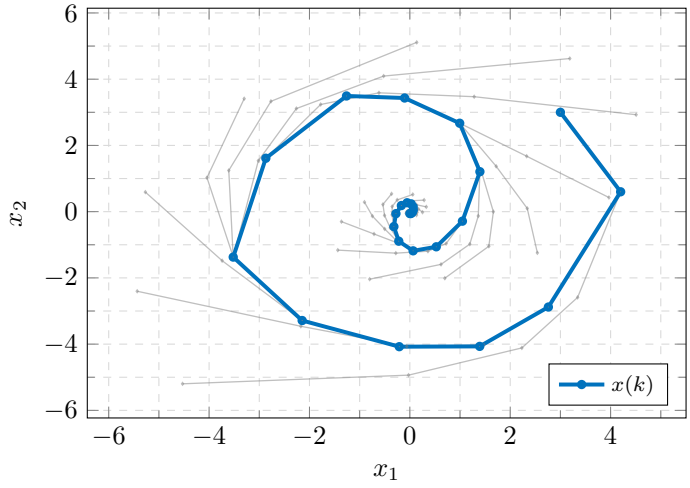


Figure 2.1: Example of regulation using model predictive control (example from [32]).

The main advatage of this method is that it reamains robust to disturbnaces and handles well constrains on the control inputs and states. However, it is computationally coslty and can present numerical convergence issues.

2.2.2 Differential Flatness Based Control

As shown in [31], the quadrotor's state and control inputs can be expressed as algebraic function of carefully selected flat outputs. Choosing the position of the center of mass $\mathbf{r} = [r_x, r_y, r_z]^\top$ and its heading angle η as outputs, the quadorotor's $\hat{\mathbf{b}}_3$ unit vector has the same direction as its thrust vector which corresponds to its acceleration,

$$\mathbf{t} = \ddot{\mathbf{r}} + [0, 0, g]^\top \tag{2.5}$$

$$\hat{\mathbf{b}}_3 = \frac{\mathbf{t}}{||\mathbf{t}||}. \tag{2.6}$$

From the heading angle, the heading vector $\mathbf{h}$ can be obtained, and subsequently the $\hat{\mathbf{b}}_\mathbf{2}$ and $\hat{\mathbf{b}}_\mathbf{1}$ axis,

$$\mathbf{h} = [\cos\eta, \sin\eta, 0]^\top \tag{2.7}$$

$$\hat{\mathbf{b}}_2 = \frac{\hat{\mathbf{b}}_3 \times \mathbf{h}}{||\hat{\mathbf{b}}_3 \times \mathbf{h}||} \tag{2.8}$$

$$\hat{\mathbf{b}}_1 = \hat{\mathbf{b}}_2 \times \hat{\mathbf{b}}_3 \tag{2.9}$$

which defines the attitude of the drone, $\mathbf{R}(\phi, \theta, \psi) = [\hat{\mathbf{b}}_\mathbf{1}, \hat{\mathbf{b}}_\mathbf{2}, \hat{\mathbf{b}}_\mathbf{3}]$. From Newton's equation of motion we have that the translational dynamics are,

$$m\ddot{r} = -mg\hat{\mathbf{e}}_3 + T_c\hat{\mathbf{b}}_3 \tag{2.10}$$

and from Euler equation, we get the rotational dynamics,

$$\boldsymbol{\tau} = \mathbf{J}\dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times \mathbf{J}\boldsymbol{\omega} \tag{2.11}$$

where $\mathbf{J}$ is the inertia matrix and $\dot{\boldsymbol{\omega}}$ are the body angluar rates. The first derivative of Equation 2.10 gives,

$$m\dddot{r} = \dot{T}_{c1}\hat{\mathbf{b}}_3 + \boldsymbol{\omega} \times u_1\hat{\mathbf{b}}_3. \tag{2.12}$$

The change in collective thurst $\dot{T}_c$ corresponds to the projection $\dot{T}_c = \hat{\mathbf{b}}_3 \cdot m\dddot{\mathbf{r}}$, substituiting in Equation 2.12, we can obtain the vector $\mathbf{h}_\omega = \boldsymbol{\omega} \times \hat{\mathbf{b}}_3$,

$$\mathbf{h}_\omega = \frac{m}{u_1}(\ddot{\mathbf{r}} - (\hat{\mathbf{b}}_3 \cdot \ddot{\mathbf{r}})\hat{\mathbf{b}}_3) \tag{2.13}$$

which allows to obtain the angular velocities. To completely define the quadrotor's state we need the angular accelerations $\dot{\boldsymbol{\omega}}$ which can be obtained from the second derivative of Equation 2.10. Hence, the control inputs are function of the flat outputs as $u_1 = T_c = m||\mathbf{t}||$, and $[u_2, u_3, u_4]^\top = \boldsymbol{\tau}$.

This property is exploited in subsection 3.2.3 to determine the attitude of the quadrotor and design a geometric controller.

2.3 Learning-Based approaches

Learning-based approaches can be understood as optimization methods in which the optimization process is done by intereacting with the environment [33]. There are two main types of RL, model-based and model-free, each have with their own advantages and drawbacks. Next we give an overview model-free and model-based RL and the most relevant algorithms in the literature of each.

2.3.1 Model-Free Reinforcement Learning

In model-free RL, neither the environment nor the dynamics of the agent are taken into account for the optimization process. Instead, the policy is optimized by gathering data from interacting with the environment. Next, we review three model-free RL approaches, Q-Learning, and two Policy Optimization methods; Policy Gradient and PPO.

Q-Learning

In Q-learning, the objective is to estimate the value of taking a given action at a particular state and store these values in a Q-table. During training, Bellman's equation is used to iteratively update the entries of the table. Once training converges, an optimal policy π^* can be obtained, since the Q-table provides the action with the highest expected return for each state [34].

$$\pi^*(s) = \arg\max_a Q^*(s, a) \tag{2.14}$$

Q-learning can be interpreted as a formulation of Dynamic Programming (DP). For simple problems, such as finding the optimal strategy to escape a maze, this approach performs well. However, for high-dimensional or complex environments, storing and updating the Q-table becomes computationally infeasible.

To overcome this limitation, Deep Q-Learning (DQN) was introduced in [35]. Instead of explicitly storing the Q-values in a table, a neural network is trained to approximate the action-value function.

Policy Gradient

In policy optimization methods, instead of learning the action-value function and deriving an optimal policy by selecting the best action at a given state, the policy is learned directly through the parameters of a neural network θ [34]. These parameters are optimized with respect to an objective function $J(\theta)$ that encodes the desired behavior of the policy.

$$J(\theta^*) = \arg\max_\theta \mathbb{E}_{\tau\sim\pi_\theta}\left[R(\tau)\right] \tag{2.15}$$

In this framework, policy optimization methods aim to find the optimal parameters θ^* such that the resulting policy $\pi_\theta(a|s)$ selects actions $a \in \mathcal{A}$ for states $s \in \mathcal{S}$ that maximize the expected return over trajectories $\tau = \{s_0, a_0, s_1, a_1, \dots\}$.

Proximal Policy Optimization

This policy optimization method was introduced in 2017 in [16] and has become a standard algorithm in RL. The algorithm adresses instability issues present in policy gradient methods. Differentiating Equation 2.15 gives,

$$\nabla_\theta J(\theta) = \mathbb{E}_{s,a\sim\pi_\theta}[\nabla_\theta \log \pi_\theta(a|s) Q^\pi(s,a)]. \tag{2.16}$$

In practice, Q^π is replaced by an advantage estimate A^π to reduce variance. Using this vanilla gradient can lead to excecissve parameter updates that destabilize the policy. Previously, this issue was adressed in [36] where the authors introduce the Trust Region Policy Optimization (TRPO) which fixes a hard KL-divergence constrain. Note that KL-divergence measures the difference between two data distributions $D_{KL}(P\|Q) = \mathbb{E}_x \log \frac{P(x)}{Q(x)}$.

$$\begin{aligned} \underset{\theta}{\text{maximize}} \quad & \mathbb{E}\left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} A_t\right] \\ \text{subject to} \quad & D_{KL}(\pi_{\theta_{old}}|\pi_\theta) \leq \delta. \end{aligned} \tag{2.17}$$

However, this method involves the calculation of the Hessian, a second-order derivative matrix which is computationally costly and complex. Instead of imposing a hard constrain the authors of [16] propose two relaxation methods.

(i) **Adaptive KL Penalty Coefficient**
The cost function becomes,

$$L^{KLPEN}(\theta) = \hat{\mathbb{E}}_t\left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}\hat{A}_t - \beta \mathrm{KL}[\pi_{\theta_{old}}(\cdot|s_t), \pi_\theta(\cdot|s_t)]\right] \tag{2.18}$$

where β controls the weight of the penalization when the new update deviates too much from the old policy parameters.

(ii) **Clipped Surrogate Objective**
Instead of penalizing the KL divergence, the probability ratio $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is clipped in the cost function.

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t\left[\min\left(r_t(\theta)\hat{A}_t, \mathrm{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right] \tag{2.19}$$

where epsilon is a hyperparameter.

The authors claim that the clipped surrogate objective approach performs better. Since PPO is usually implemented as an actor-critic algorithm a loss term for the critic, $L_t^{VF}(\theta)$, is incorporated, and to encourage exploration, an entropy bonus, $S[\pi_\theta](s_t)$, is added.

Hence, the final PPO loss is,

$$L_t^{CLIP+VF+S}(\theta) = \hat{\mathbb{E}}_t\left[L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t)\right] \tag{2.20}$$

whera c_1 and c_2 are weighting factors.

2.3.2 Model-Based Reinforcement Learning

Model-based RL uses infromation of a model to optimize the policy parameters. The model can be predifined, or it can be learned by a neural newrok.

Given The Model

When the model is know, that is to say, the state equation $\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k)$. The RL architecture can be straightforward, the gradient from the cost function J to the policy parameters can be computed directly by appliyng the chain rule. And the policy is optimized BPTT of the accumulated gradient. In section section 2.4, we explain in more detail how the optimization porcess works as this is the method we use in our methodology to learn the drone racing policy.

Having knowledge of the model increseas the sample efficeny. However, if the model is not suitable error can accumulate leading to suboptimal performance. A perfect model is not necessary, but it has to capture the important behaviours of the system. Additionally, neural newteorks tend to overfit to the training data distribution. To prevent ovrefitting, it is recommended to radnomize model parameters, see section 3.3.

Learn The Model

An alternative approach involves learning the state transition dynamics directly from environmental interactions to estimate the state equation $\hat{\mathbf{x}}_{k+1} = \hat{f}_\theta(\mathbf{x}_k, \mathbf{u}_k)$. A notable framework in this domain is the Dreamer architecture [20]. Rather than modeling the world in the raw observation space, Dreamer utilizes an encoder to map observations $\mathbf{o}_k$ into stochastic latent representations $\mathbf{z}_k$. The system's dynamics are then captured by a recurrent state-space model: $\mathbf{h}_{k+1} = f(\mathbf{h}_k, \mathbf{z}_k, \mathbf{u}_k)$. This approach offers two primary advantages: the recurrent state provides essential temporal memory, while the mapping to a stochastic latent space facilitates the processing of high-dimensional inputs, such as raw images.

Once the world model is learned, an actor-critic agent can be trained entirely within this learned environment. This paradigm shares conceptual similarities with Model Predictive Control (MPC), as it allows the agent to account for the impact of current actions on future states. In [19], the authors employ a similar architecture to train an Autonomous Drone Racing (ADR) policy that maps pixels directly to motor commands. Their method demonstrated champion-levlel performance; however, a drawback of this method is its sample inefficiency, as the authors report that training can require up to 50 hours.

2.4 Differentiable Optimization

Under the paradigm of differentiable optimization, we no longer treat the quadrotor model as a "black-box". The main idea of this thesis is to leverage the simulator's automatic differentiation engine to compute the exact analytical gradient of the cumulative loss J_k with respect to the control policy network. In this section we provide an overview on how the optimization process works.

2.4.1 Computational Graph and Backpropagation

The gradient of the cummulative stage cost function J_k with respect to the network parameters θ is given by the chain rule as,

$$\nabla_\theta J_k = \frac{1}{B}\frac{1}{H}\sum_{i=1}^{B}\sum_{h=k}^{k+H-1}\frac{\partial \mathcal{L}_h}{\partial \mathbf{x}_h}\frac{\partial \mathbf{x}_h}{\partial \mathbf{u}_{h-1}}\frac{\partial \mathbf{u}_{h-1}}{\partial \theta} \tag{2.21}$$

where $\frac{\partial \mathcal{L}_h}{\partial \mathbf{x}_h}$ is the gradient of the stage loss with respect to the state, $\frac{\partial \mathbf{x}_h}{\partial \mathbf{u}_{h-1}}$ encodes the change in the dynamics regarding the applied controls, and $\frac{\partial \mathbf{u}_{h-1}}{\partial \theta}$ is the gradient of the control output with respect to the policy parameters. Since the gradient is accumulated through time due to the recursive dependence of the state on previous states and control inputs, this procedure is known as BPTT.

To compute this gradient analytically, an automatic differentiation engine is used. Many ML frameworks, such as PyTorch [6], TensorFlow [8], and JAX [7], among others, provide autodiff capabilities. At present, JAX is regarded as a high-performance framework for numerical ML computing. However, for this simulator, PyTorch has been selected due to its intuitive interface, extensive documentation, and lower implementation complexity.

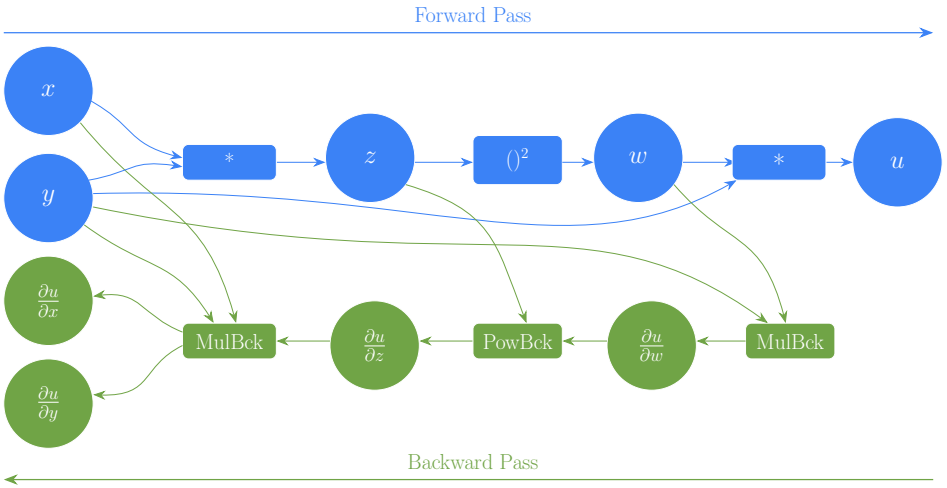


Figure 2.2: Computational graph of the numerical operations of the forward and backward passes.

To illustrate how gradients are computed, consider the next function $f(x, y) = y(xy)^2$. Its gradient is $\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} & \frac{\partial f}{\partial y} \end{bmatrix} = \begin{bmatrix} 2xy^3 & 3x^2y^2 \end{bmatrix}$ and evaluating at $x = 1$ and $y = 2$, we get $\nabla f(1, 2) = \begin{bmatrix} 16 & 12 \end{bmatrix}$. In PyTorch, this gradient is computed using a computational graph which is constructed during the forward pass. Each mathematical operation is stored as a node in this graph. During backporopagation, PyTorch applies automatic differentiation to traverse the graph from the output back to the input and applying the chain rule to accumulate gradients. The computational graph shown in Figure 2.2 represents this process and Algortihm 1 presents the pesudocode. In practice, the calculation of the gradients is done simply by calling **u.backward()** in PyTorch.

Adittionally, PyTorch allows to custimoize the computational graph in a manner that the forward and backward pass do not need to be identical. As shown in Figure 3.6, this feature is leveraged to skip the motor dynamics during the backward pass. This is done to obtain cleaner gradients since the derivative of Equation 3.10 is very sensitive due to the small values of the constant $a_0 \sim 10^{-7}$ and the square root.

Algorithm 1 Example Backpropagation for the function $u(x, y) = y(xy)^2$

Require: $x \in \mathbb{R},\ y \in \mathbb{R}$
Ensure: $\frac{\partial u}{\partial x},\ \frac{\partial u}{\partial y}$
— **Forward Pass (Build Computational Graph)** —
$x \leftarrow$ `tensor`(1.0, `requires_grad=True`) ▷ Leaf node
$y \leftarrow$ `tensor`(2.0, `requires_grad=True`) ▷ Leaf node
$z \leftarrow x \times y$ ▷ Node: `MulBackward`
$w \leftarrow z^2$ ▷ Node: `PowBackward`
$u \leftarrow w \times y$ ▷ Output Node: `MulBackward`
— **Backward Pass (Reverse-Mode Autodiff)** —
Initialize $\frac{\partial u}{\partial u} \leftarrow 1$
$\frac{\partial u}{\partial w} \leftarrow \frac{\partial u}{\partial u} \cdot y = 1 \cdot 2 = 2$ ▷ Backprop through $u = w \cdot y$
$\frac{\partial u}{\partial y} \mathrel{+}= \frac{\partial u}{\partial u} \cdot w = 1 \cdot 4 = 4$ ▷ Accumulate y gradient from this node
$\frac{\partial u}{\partial z} \leftarrow \frac{\partial u}{\partial w} \cdot 2z = 2 \cdot 2 = 4$ ▷ Backprop through $w = z^2$
$\frac{\partial u}{\partial x} \leftarrow \frac{\partial u}{\partial z} \cdot y = 4 \cdot 2 = 8$ ▷ Backprop through $z = x \cdot y$
$\frac{\partial u}{\partial y} \mathrel{+}= \frac{\partial u}{\partial z} \cdot x = 4 \cdot 1 = 4$ ▷ Accumulate y gradient from this node
— **Results** —
$\frac{\partial u}{\partial x} = 16, \quad \frac{\partial u}{\partial y} = 4 + 4 + 4 = 12$

2.4.2 Optimization Process

Given the gradient $\nabla_\theta J$ computed via BPTT the policy parameters are updated using the Adaptive Moment Estimation (Adam) optimizer [37]. Adam is a variant of Stochastic Gradient Descent (SGD), in standard SGD the model parameters are updated proportionnally to the gradient in its opposite direction,

$$\theta_{k+1} = \theta_k - \alpha \nabla_\theta J_k \tag{2.22}$$

where α inidcates the learning rate. The main disatvantages of standard SGD is that near the optimum, the gradient decreases rquiring more steps to converge and each parameter is updated equally. Adam solves this issues by combining two other variations of SGD: Momentum and Root Mean Square Propagation (RMSProp).

- Momentum accumulates an avarage of past gradients sustaining progress along the directions of consistent descent and dampening oscillations in high curvature directions.
- RMSProp avarages the square of the gradients, which are used to scale the learning rate of each parameter individually, hence, parmeters with high historical gradients revice smaller updates than those with smaller gradientes

Adam unifies these two concepts through the first and second moment estimates, m and v_k respectively, which are updated as,

$$m_k = \beta_1 m_{k-1} + (1 - \beta_1) \nabla_\theta J_{k-1} \tag{2.23}$$

$$v_k = \beta_2 v_{k-1} + (1 - \beta_2)(\nabla_\theta J_{k-1})^2 \tag{2.24}$$

where β_1 and β_2 are hyperparameters usually set to 0.9 and 0.999 respectively. Since both moments are initialized at zero $m_0 = v_0 = 0$, the estimates are biased towards zero in the early iterations of training. For this reason, each moment is normalized to,

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^k} \tag{2.25}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^k}. \tag{2.26}$$

The final prameter update is given by,

$$\theta_{k+1} = \theta_k - \alpha \frac{\hat{m}_k}{\sqrt{\hat{v}_t} + \epsilon} \tag{2.27}$$

where $\epsilon \ll 1$ is a small constant introduced for numerical stability.

Chapter 3

Methodology

This chapter presents the methodology used in this thesis to train the neural network policies. In **??**, we adress the core elements of the simulation architecture, define the quadrotor model, and explain how the Simulation-to-Reality (Sim2Real) gap is minimized by using DR. Furthermore, we introduce the implemented control abstraction levels. Section 3.4, outlines the training objectives used for ADR. Lastly, **??** details how differentiability is leveraged to optimize the policies in order to minimize the training task's cost function.

3.1 Quadrotor Model

In the simulated environments, the quadcopter is modeeld as a 6 Degrees of Freedom (DoF) rigid body. The inertial world frame $\mathcal{W}$ is defined by the fixed orthonormal basis $\{\hat{\mathbf{e}}_1, \hat{\mathbf{e}}_2, \hat{\mathbf{e}}_3\}$ while the non-inertial body-fixed frame $\mathcal{B}$ by the orthonormal basis $\{\hat{\mathbf{b}}_1, \hat{\mathbf{b}}_2, \hat{\mathbf{b}}_3\}$, which is attached to the vehicle's center of mass. The state of the Unmanned Aerial Vehicle (UAV) is given by $\mathbf{x} = [\mathbf{p}, \mathbf{v}, \mathbf{q}, \boldsymbol{\omega}, \boldsymbol{\Omega}]^\top$ where the position $\mathbf{p} \in \mathbb{R}^3$, velocity $\mathbf{v} \in \mathbb{R}^3$, and acceleration are represented in the world frame $\mathcal{W} \in \mathbb{R}^3$. Conversely, the body rates $\boldsymbol{\omega} \in \mathbb{R}^3$ are represented in the body frame $\mathcal{B} \in \mathbb{R}^3$ and $\boldsymbol{\Omega} \in \mathbb{R}^4$ is a vector of scalar values indicating the rotor speeds. The relationship between the two frames is given by the rotation mapping $\mathbf{b}_i = \mathbf{R}\mathbf{e}_i$, where $\mathbf{R} \in SO(3)$.

Thus, the kinematics and dynamics of the quadcopter are:

$$\dot{\mathbf{p}}^{\mathcal{W}} = \mathbf{v}^{\mathcal{W}} \tag{3.1}$$

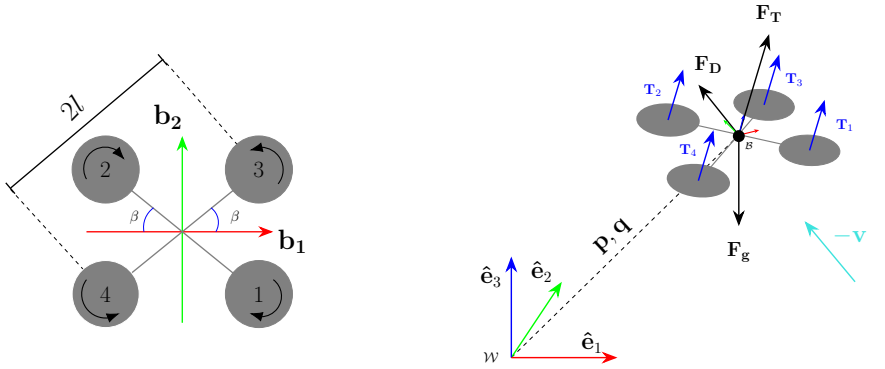
$$\dot{\mathbf{q}}^{\mathcal{W}} = \tfrac{1}{2}\, \mathbf{q}^{\mathcal{W}} \odot \begin{bmatrix} 0 \\ \boldsymbol{\omega}^{\mathcal{B}} \end{bmatrix} \tag{3.2}$$

$$\dot{\mathbf{v}}^{\mathcal{W}} = \frac{\mathbf{R}(\mathbf{q})}{m} \left(\mathbf{F}_T^{\mathcal{B}} + \mathbf{F}_D^{\mathcal{B}}\right) + \mathbf{g}^{\mathcal{W}} \tag{3.3}$$

$$\dot{\boldsymbol{\omega}}^{\mathcal{B}} = \mathbf{J}^{-1} \left(\boldsymbol{\tau}^{\mathcal{B}} - \boldsymbol{\omega}^{\mathcal{B}} \times \mathbf{J}\boldsymbol{\omega}^{\mathcal{B}}\right) \tag{3.4}$$

$$\dot{\boldsymbol{\Omega}} = \frac{1}{k_{\mathrm{m}}} \left(\boldsymbol{\Omega}_{\mathrm{cmd}} - \boldsymbol{\Omega}\right) \tag{3.5}$$

In these expressions, $\odot$ denotes the quaternion multiplication operator, $\mathbf{R}(\mathbf{q})$ represents the rotation matrix mapping the body frame $\mathcal{B}$ to the world frame $\mathcal{W}$. The vectors $\mathbf{F}_T^{\mathcal{B}}$ and $\mathbf{F}_D^{\mathcal{B}}$ correspond to the collective thrust and drag forces, respectively. Finally, $\mathbf{J}$ is the diagonal inertia matrix, k_m is the motor time constant, and $\mathbf{g}$ is the gravity vector.



(a) Diagram of the motor layout.

(b) Diagram of the forces acting upon the quadrotor.

Figure 3.1: Quadrotor configuration and force diagram.

In the body frame $\mathcal{B}$, the individual motor thrusts $T_i,\ i = 1, .., 4$, are assumed to be perpendicular to the $\{\hat{\mathbf{b}}_1, \hat{\mathbf{b}}_2\}$ plane. The magnitudes of collective thrust F_T, and body torques $\boldsymbol{\tau}^{\mathcal{B}}$, can be obtained from Equation 3.6. To be strictly rigourous, the torques produced by the rotor's angular acceleration and gyroscopic effects should be accounted for. Where $\mathbf{\Gamma}$ is the allocation matrix. Nonetheless, these effects produced by the rotor speeds, can be neglected the mapping simplifies to,

$$\begin{bmatrix} F_T \\ \tau_{b1} \\ \tau_{b2} \\ \tau_{b3} \end{bmatrix} = \overbrace{\begin{bmatrix} 1 & 1 & 1 & 1 \\ -l\sin\beta & l\sin\beta & l\sin\beta & -l\sin\beta \\ -l\cos\beta & l\cos\beta & -l\cos\beta & l\cos\beta \\ -c_{tf} & -c_{tf} & c_{tf} & c_{tf} \end{bmatrix}}^{\mathbf{\Gamma}} \begin{bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \end{bmatrix} \tag{3.6}$$

where l is the arm length, β is the arm angle, and c_{tf} the linear torque constant. The individual motror thrusts T_i, can be obtained from the rotor speeds Ω_i with the thrust map coefficents, $\mathbf{k_T} = [a_1,\ a_2,\ a_3]$ which can be obtained experimentally by fitting a quadratic curve to the rotor speed versus thrust data.

$$T_i = \mathbf{k_T} \begin{bmatrix} \Omega_i^2 \\ \Omega_i \\ 1 \end{bmatrix} = \begin{bmatrix} a_0 & a_1 & a_2 \end{bmatrix} \begin{bmatrix} \Omega_i^2 \\ \Omega_i \\ 1 \end{bmatrix} \tag{3.7}$$

For convinence, Equation 3.7 is simplified to $T_i = a_0\Omega_i^2$.

To complete the quadrotor model, the aerodynamic drag force $\mathbf{F}_D$ is incorporated using a simplified quadratic formulation. Other aerodynamic effects, such as rotational drag and ground interaction, are neglected. The drag force is expressed as:

$$\mathbf{F}_D^{\mathcal{B}} = -\tfrac{1}{2}\rho\|\mathbf{v}^{\mathcal{B}}\|\mathbf{C_D}\mathbf{A}\mathbf{v}^{\mathcal{B}} \tag{3.8}$$

where ρ denotes the air density, $\mathbf{A} = \text{diag}(A_{b1}, A_{b2}, A_{b3})$ represents the reference area along each body axis, and $\mathbf{C_D} = \text{diag}(C_{D_{b1}}, C_{D_{b2}}, C_{D_{b3}})$ is a diagonal matrix of non-dimensional drag coefficients. The velocity must be expressed in the body frame, obtained as $\mathbf{v}^{\mathcal{B}} = \mathbf{R}(\mathbf{q})^\top \mathbf{v}^{\mathcal{W}}$.

3.2 Control Abstraction

The control abstraction refers to the modality of the policy outputs used to control the quadrotor. The choice of control abstraction affects both the performance of the controller for a given task and the sample efficiency during training. For this reason, it is worthwhile to investigate and implement multiple control abstractions.

In this thesis, three control modalities are considered. The policy output layer employs a sigmoid activation function for all control modalities, ensuring that the action space is bounded, which facilitates stable learning. The lowest level of abstraction consists of directly outputting Single Rotor Thrust (SRT) for each motor. An intermediate level of abstraction outputs the collective thrust and the desired body rates, which are subsequently mapped to individual rotor thrusts through a control allocation matrix. The highest level of abstraction outputs desired linear velocities and yaw rate, which are converted into a wrench using an SO(3) geometric controller [38], and subsequently mapped to rotor thrusts in a manner analogous to the collective thrust and body rate formulation. The mathematical formulation of these transformations is presented next.

3.2.1 Single Rotor Thrust

When the SRT control mode is choosen, the only transformation to be applied is scaling the policy output $\boldsymbol{u} \in [0, 1] \in \mathbb{R}^4$ to the actuator limits, $\mathbf{T}_{\text{cmd}} \in [T_{\min}, T_{\max}]$. Therefore, the transformation is as simple as;

$$\boldsymbol{T}_{\text{cmd}} = \boldsymbol{u}\,(T_{\max} - T_{\min}) + T_{\min} \tag{3.9}$$

Once the scaling is applied, to better emulate the behaviour of a real motor, a first order model with rotor speed saturation is implemented. First, the commanded motor thrust $\boldsymbol{T}_{\text{cmd}}$ are converted to commaned rotor speed. This mapping can be obtained experimentally by fitting a quadratic curve to the experimental data. In most cases the following mapping apprixmation is good enough.

$$\boldsymbol{T}_{\text{cmd}} \approx k\boldsymbol{\Omega}^2_{\text{cmd}} \tag{3.10}$$

Then the commanded rotor speeds $\boldsymbol{\Omega}_{\text{cmd}}$, are passed through the following first order motor model Equation 3.11, and saturated according to the maximum rotor spee rate, $\dot{\Omega}_{\text{cmd}}$ given by the manufacturer.

$$H(s) = \frac{1}{k_m + s} \tag{3.11}$$

Finally, the rotor speeds obtained after the described process are mapped again to single rotor thurst values using Equation 3.10 and then converted to collective force and torques with Equation 3.6, to calculate the next state through the quadrotor dynamics model. Modeling the motor dynamics presents more realistic data which the policy will encounter at deployement. However, for training purposes it has been decided to skip the motor dynamics during the backporopagation pass as including them, significantly destabilizes the gradients computation. Particularly, the mapping of Equation 3.10 produces high variations on the gradient, degrading the training performance and sample efficiency.

3.2.2 Collective Thrust and Body Rates

When Collective Thrust and Body Rates (CTBR) control mode is selected, the normalized outputs of the policy $\boldsymbol{u} \in [0,1] \in \mathbb{R}^4$, are mapped to collective thrust and body rates. The scaling is as follows,

$$T_{c,\text{cmd}} = u_1(T_{\max} - T_{\min}) + T_{\min} \tag{3.12}$$

$$\omega_{i,\text{cmd}} = (2u_{i+1} - 1)\omega_{i,\max}, \quad i \in \{1,2,3\} \tag{3.13}$$

where $\omega_{i,\max}$ indicates the maximum body rate of the respective axis, (roll, pitch, yaw). Once the rates are calculated, they can be converted to torque values,

$$\boldsymbol{\tau}_{\text{cmd}} = k_p \frac{(\boldsymbol{\omega}_{\text{cmd}} - \boldsymbol{\omega})}{dt} J \tag{3.14}$$

aditionally, a proportinal gain k_p can be introduced to adjust the stiffness of the conversion. Given the commanded collective thrust and torques, the motor thursts can be obtained from **??** and then the same procudeure of subsection 3.2.1 is applied during the forward and backward passes.

3.2.3 Linear Velocities and Heading Rate

The highest level of abstraction that is implement consists on commanding Linear Velocities and Heading Rate (LVHR). With this control mode, the policy outputs are scaled as,

$$v_{i,\text{cmd}} = (2u_i - 1)v_{i,\max}, \quad i \in \{1,2,3\} \tag{3.15}$$

$$\omega_{3,\text{cmd}} = \dot{\eta}_{\text{cmd}} = (2u_4 - 1)\dot{\eta}_{\max} \tag{3.16}$$

where ψ denotes the yaw angle and $v_{i,\text{cmd}}$ represents the commanded linear velocity along each body axis. Mapping these control commands to thrust commands is less straightforward than in the previous two methods. Instead, the collective thrust and torques are obtained by applying a transformation in the Special Orthogonal group SO(3). The transfomations proceed in the following manner.

First, the acceleration vector is computed as

$$\mathbf{a}_{\text{cmd}} = k_v(\mathbf{v}_{\text{cmd}} - \mathbf{v}) + g\hat{\mathbf{b}}_3 \tag{3.17}$$

where k_v is a proportional gain on the velocity. Note that the acceleration along the third body axis is compensated for gravity; $\hat{\mathbf{b}}_3$ denotes the unit vector along this axis. Then, force vector is the UAV's mass times the commanded acceleration vector, and the commanded collective thrust vector is the projection of the force vector onto the $\hat{\mathbf{b}}_3$ axis.

$$\mathbf{F} = m\mathbf{a}_{\text{cmd}} \tag{3.18}$$

$$T_{c,\text{cmd}} = \mathbf{F} \cdot \hat{\mathbf{b}}_3 \tag{3.19}$$

To obtain the commanded bodytorques, the heading vector is obtained by projecting the yaw angle into the world plane defined by span$\{\hat{e}_1, \hat{e}_2\}$. The heading vector, serves an auxiliary vector to compute $\hat{\mathbf{b}}_2$ and $\hat{\mathbf{b}}_1$.

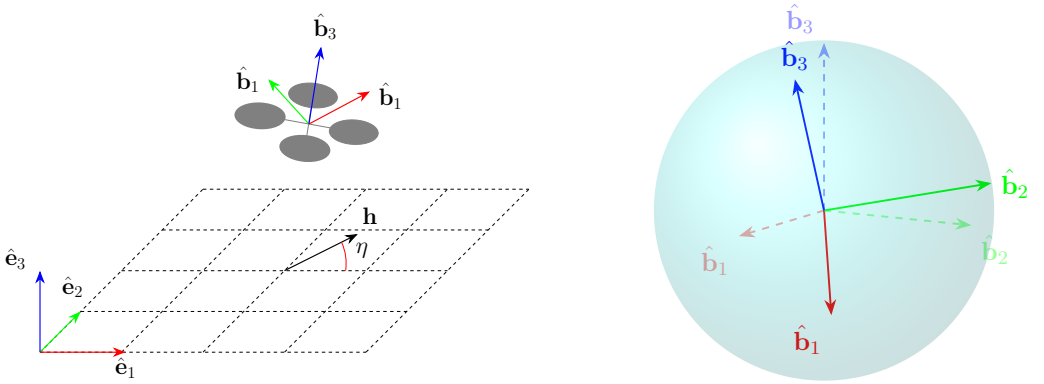
$$\mathbf{h} = \begin{bmatrix} \cos\eta & \sin\eta & 0 \end{bmatrix}^\top \tag{3.20}$$

$$\hat{\mathbf{b}}_2 = \frac{\hat{\mathbf{b}}_3 \times \mathbf{h}}{\|\hat{\mathbf{b}}_3 \times \mathbf{h}\|} \tag{3.21}$$

$$\hat{\mathbf{b}}_1 = \hat{\mathbf{b}}_2 \times \hat{\mathbf{b}}_3 \tag{3.22}$$

In this way, the commanded orientation is defined by $\mathbf{R}_{\text{cmd}} = \begin{bmatrix} \hat{\mathbf{b}}_1 & \hat{\mathbf{b}}_2 & \hat{\mathbf{b}}_3 \end{bmatrix}$ and the error between the commanded orientation and the current orientation is computed as in [38].

$$\mathbf{e}_R = \frac{1}{2}\left(\mathbf{R}_{\text{cmd}}^\top \mathbf{R} - \mathbf{R}^\top \mathbf{R}_{\text{cmd}}\right)^\vee \tag{3.23}$$



(a) Projection of the drone's heading angle η.

(b) Representation of the $SO(3)$ rotation space.

Figure 3.2: Geometric interpretation of Equations 3.12–3.15

The complete derivation of Equation 3.23 can be found in the original paper, [38]. To provide intuition, note that if $\mathbf{R}_{\text{cmd}} = \mathbf{R}$, then $\mathbf{R}_{\text{cmd}}^\top \mathbf{R} = \mathbf{I}$ and the error is zero. Otherwise, the matrix $\mathbf{R}_{\text{cmd}}^\top \mathbf{R}$ represents the relative rotation between the current and desired orientations. The skew-symmetric part of this matrix, given by $\frac{1}{2}(\mathbf{R}_{\text{cmd}}^\top \mathbf{R} - \mathbf{R}^\top \mathbf{R}_{\text{cmd}})$, encodes the axis and magnitude of the rotation error. The vee map $(\cdot)^\vee$ then converts this skew-symmetric matrix into a vector in $\mathbb{R}^3$, corresponding to the rotation error expressed in the Lie algebra $\mathfrak{so}(3)$, that is to say, $^\vee : \mathfrak{so}(3) \to \mathbb{R}^3$. Then, the factor $\frac{1}{2}$ ensures consistency with the standard definition of the vee operator and avoids double counting of the off-diagonal terms.

Adittionally, an error on the angular velocity of the rotation is introduced as,

$$\mathbf{e}_{\boldsymbol{\omega}} = \boldsymbol{\omega} - \mathbf{R}^\top \mathbf{R}_{\text{cmd}} \boldsymbol{\omega}_{\text{cmd}} \tag{3.24}$$

where $\boldsymbol{\omega}_{\text{cmd}} = \begin{bmatrix} 0 & 0 & \dot{\eta}_{\text{cmd}} \end{bmatrix}^\top$. Note that the error between the desired rotational velocity and the current rotational velocity line on different tangent planes to the unit sphere, $\dot{\mathbf{R}} \in$

$\mathrm{T}_{\mathbf{R}}SO(3)$ and $\dot{\mathbf{R}}_{\text{cmd}} \in \mathrm{T}_{\mathbf{R}_{\text{cmd}}}SO(3)$. For this reason, as explained in [38], $\dot{\mathbf{R}}_{\text{cmd}}$ is transformed into a vector in the $\mathrm{T}_{\mathbf{R}}SO(3)$ space to be able to compare it to $\dot{\mathbf{R}}$ and obtain the error; Equation 3.24. Finally, the torque is set by,

$$\boldsymbol{\tau}_{\mathbf{cmd}} = -k_R\mathbf{e}_{\mathbf{R}} - k_{\omega}\mathbf{e}_{\boldsymbol{\omega}} + \boldsymbol{\omega} \times \mathbf{J}\boldsymbol{\omega} - \mathbf{J}(\hat{\boldsymbol{\omega}}\mathbf{R}^{\top}\mathbf{R}_{\text{cmd}}\boldsymbol{\omega}_{\text{cmd}} - \mathbf{R}^{\top}\mathbf{R}_{\text{cmd}}\dot{\boldsymbol{\omega}}_{\text{cmd}}) \tag{3.25}$$

where a gyroscopic and feed forward terms are added to the rotation and rotational speed errors. The *hat map* $\hat{\cdot}: \mathbb{R}^3 \to \mathfrak{so}(3)$ is defined by the condition that $\hat{x}y = x \times y$ for all $x, y \in \mathbb{R}^3$. In this manner, the commanded collective thrust Equation 3.19, and commanded torques Equation 3.25, can be mapped to single rotor thrust commands to apply the same procedure of the last control modes during the forward and backward passes.

3.2.4 Linear Velocity Heading Rate and Gains

Finally, the last control mode implemented is an augmentation of the LVHR controller. In this formulation, the policy outputs high-level commands in the form of desired linear velocities and yaw rate, rather than directly commanding individual rotor thrusts. This higher level of abstraction simplifies the learning process, as the policy does not need to implicitly learn how differential rotor thrusts generate torques, since this mapping is handled by the geometric controller. However, this approach does not necessarily minimize the loss function, as the policy performance can be significantly constrained by the tuning of the controller gains. For this reason, it is of interest to investigate how performance is affected when the policy is allowed to output the controller gains in addition to the velocity and yaw rate commands.

Thus, when this control mode is selected, the policy outputs are, the same as Equation 3.15, Equation 3.24, and the gains $\{k_v, k_R, k_\omega\}$ are included by simply scaling the policy outputs to the maximum allowed gains.

$$k_{v,cmd} = u_5 k_{v,\max} \tag{3.26}$$

$$k_{R,cmd} = u_6 k_{R,\max} \tag{3.27}$$

$$k_{\omega,cmd} = u_7 k_{\omega,\max} \tag{3.28}$$

3.3 Domain Randomization

To improve the transferability of the policy and to prevent the it from ovrefitting to a specific environment, DR is implemented in our methodology as it is well-stabilshed strategy in the field.

A domain can be defined as a specific subset of all possible Partially Observable Markov Decision Process (POMDP) environments. While a complete POMDP is formally characterized by an 8-tuple [39], we focus on a reduced 6-tuple representation that captures the essential elements for our domain definition. A standard domain is then characterized by $\mathcal{D} = (\mathcal{S}, \mathcal{A}, \mathcal{O}, P, O, R)$ where:

- $\mathcal{S}$: denotes a set of states called the state space.
- $\mathcal{A}$: is the set of actions called the action space.
- $\mathcal{O}$: is the set of observations called the observation space.

- $\mathbf{P}(\mathbf{s}'|\mathbf{s},\mathbf{a})$: is the probability that a given action $a \in A$, at state $s \in S$ will lead to a new state $s' \subset S$.
- $\mathbf{O}(\mathbf{o}|\mathbf{s}',\mathbf{a})$: is the observation probability, representing the probability of observing $o \in \mathcal{O}$ after taking action a and transitioning to state s'.
- $\mathbf{R}(\mathbf{s}',\mathbf{s},\mathbf{a})$: is the immediate reward, or expected inmediate reward, received for transitioning from s to s' after performing action a.

From the source domain, $\mathcal{D}_{\xi_s}$, (i.e. simulation), we can control a set of N parameters that define a configuration vector, $\boldsymbol{\xi} = [\xi_i, \xi_{i+1}, ..., \xi_N]$. These parameters will affect the transition probability $\mathbf{P}(\mathbf{s}'|\mathbf{s},\mathbf{a})$ of the domain. Examples of ξ_i can include:

Physical properties	ξ_{mass}, ξ_{inertia}, ξ_{drag}, ξ_{gravity}
Perception properties	$\xi_{\text{sensor\_noise}}$, $\xi_{\text{sensor\_latency}}$
Viual properties	ξ_{lighting}, ξ_{texture}, ξ_{fov}
Scene properties	$\xi_{\text{gate\_pos}}$, $\xi_{\text{gate\_size}}$, , $\xi_{\text{gate\_num}}$

When DR is applied, each configuration $\boldsymbol{\xi}$ is sampled from a bounded randomization space Ξ with a probability distribution. That is to say, $\boldsymbol{\xi} \in \Xi \subset \mathbb{R}^N$, $\boldsymbol{\xi} \sim p(\boldsymbol{\xi})$ where $\xi_i \in [\xi_i^{\text{low}}, \xi_i^{\text{high}}]$. At the start of every training episode, a new environment insance is created by sampling a new configuration vector $\boldsymbol{\xi}$ from the distribution $p(\boldsymbol{\xi})$. Therefore, the policy will interact with a new POMDP environment, a new domain $\mathcal{D}_{\boldsymbol{\xi}}$. This way, data from a vareity of environments that otherwise would reamain unexplored can be collected.

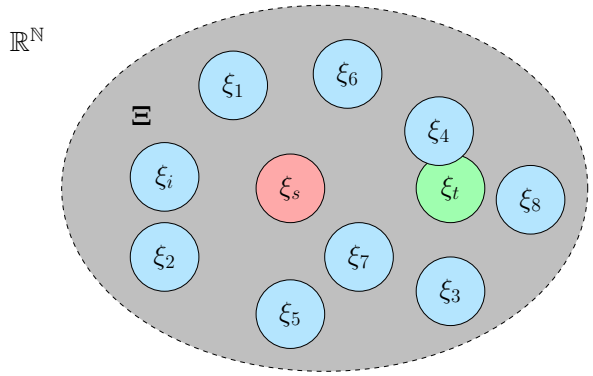


Figure 3.3: In red, the source domain. In light blue, ranodimized instances of the source configuration. In green, the target domain configuration.

Hereby, during training, the neural network weights and biases θ are optimized to minimize the expected loss across a distribution of randomized domains [40]. With the aim of finding the optimal, θ^* that will lead to the optimal policy π^*_θ.

As illustrated in Figure 3.3, the policy is exposed to a variety of domains to promote generalization and improve robustness, increasing the likelihood of successful transfer to the real environment. However, this strategy further reduces the sample efficiency of RL and increases training time. Therefore, it is essential to properly bound the randomization space Ξ, so that it encompasses only the minimum necessary range to include the target domain.

In order to efficently bound this randomization space while still allowing for enough randomness a two stage randomization is implement. Inspired by [41], first a design informed randomization is applied. This ensures that sensible variations of the nominal platform are predominant in the randomization space. Thus avoiding that unfeasible designs such as a quadrotor with unsensible Thrust to Weight Ratio (TWR) (i.e. TWR$<$ 1). The authors of

[41] notice that quadrotor designs, in general, follow a similar pattern where the quadrotors armlength l is strongly correlated to its mass m, inertia $\mathbf{J}$, drag coefficents $\mathbf{C_D}$ and other physical parameters.

Hence, they propose to sample a size factor $c \sim \mathcal{U}(0, 1)$ and scale the quadrotor's physcial parameters as listed below;

$$l = c(l_{\max} - l_{\min}) + l_{\min} \tag{3.29}$$

$$m = \frac{l^3 - l_{\min}^3}{l_{\max}^3 - l_{\min}^3}(m_{\max} - m_{\min}) + m_{\min} \tag{3.30}$$

$$\mathbf{J} = \frac{l^5 - l_{\min}^5}{l_{\max}^5 - l_{\min}^5}(\mathbf{J}_{\max} - \mathbf{J}_{\min}) + \mathbf{J}_{\min} \tag{3.31}$$

$$\mathbf{C}_D = \frac{l^2 - l_{\min}^2}{l_{\max}^2 - l_{\min}^2}(\mathbf{C}_{D_{\max}} - \mathbf{C}_{D_{\min}}) + \mathbf{C}_{D_{\min}} \tag{3.32}$$

TODO :Add the rest of the params

Following the scaling process, these parameters which comprisise the following configuration vector

$$\boldsymbol{\xi} = \begin{bmatrix} l & m & \mathbf{J} & \mathbf{C}_D \end{bmatrix} \tag{3.33}$$

are subjected to additional noise. Each parameter is multiplied by a scaling factor $\eta \sim \mathcal{U}(1 - p, 1 + p)$, where $p = 0.2$. This specific uncertainty range was selected based on experimental results from a previous project, as it demonstrated good transferability, nonetheless it can be considered a hyperparameter of the training process and be tunned according to the situation.

3.4 Training Task

The training objectives considered for this thesis aim to provide an approximate solution to the Optimal Control Problems (OCPs), formulated in subsection 2.1.3. To adress these problems, we use the proposed learning framework detailed in **??**. Unlike traditional solvers that rely on **QP-ing!** (**QP-ing!**) or model-free RL which treats the system's dynamics as a black-box, we leverage the simulator's automatic differentiation enginge to compute the analytical gradient from the cost function to the policy parameters.

Following this method, to achieve near optimum results, the training environment and policy architecture has to be carfuly desigend. In our simulation, we formulate them for the trajectory tracking tasks is the next manner.

3.4.1 Trajectory Tracking

For the trajectory tracking task, the objective is to minimize a cumulative loss function over a finite horizon of length H, defined with respect to a reference state $\mathbf{x}_k^{\text{ref}}$ and the quadrotor state $\mathbf{x}_k$. The stage loss at each timestep k, denoted as $\mathcal{L}(\mathbf{x}_k^{\text{ref}}, \mathbf{x}_k)$, is formulated as a weighted sum of position, velocity, and attitude alignment errors, together with a penalization of the body rates to improve stability during early training.

$$\mathcal{L}(\mathbf{x}_k^{\text{ref}}, \mathbf{x}_k) = \lambda_p \|\mathbf{p}_k^{\text{ref}} - \mathbf{p}_k\|_2 + \lambda_v \|\mathbf{v}_k^{\text{ref}} - \mathbf{v}_k\|_2 + \lambda_a \left(1 - \hat{\mathbf{b}}_{3,k} \cdot \mathbf{z}_k\right) + \lambda_\omega \|\boldsymbol{\omega}_k\|_2^2 \tag{3.34}$$

The desired thrust direction $\mathbf{z}_k$ is defined as:

$$\mathbf{z}_k = \frac{\mathbf{a}_k^{\text{ref}} + \mathbf{g}}{\|\mathbf{a}_k^{\text{ref}} + \mathbf{g}\|_2} \tag{3.35}$$

Since the simulator is batched, i.e., multiple environments run in parallel, the total cumulative cost over the horizon is defined as:

$$J_k = \frac{1}{B}\frac{1}{H}\sum_{i=1}^{B}\sum_{h=k}^{k+H-1} \mathcal{L}\left(\mathbf{x}_{h,i}^{\text{ref}}, \mathbf{x}_{h,i}\right) \tag{3.36}$$

where k denotes the initial timestep of the current horizon, H is the horizon length, and B is the number of parallel environments. At the end of each horizon, the gradient of the cumulative cost in Equation 3.36 with respect to the policy parameters is backpropagated through time for optimization, as described in section 2.4.

To improve numerical stability and avoid exploding gradients, training episodes are terminated when predefined conditions are met. This prevents the accumulation of large errors, which can otherwise lead unstable learning dynamics. For this task, episode termination is based solely on the position tracking error. Specifically, an episode is terminated when the Euclidean distance between the quadrotor position and the reference trajectory exceeds a predefined threshold $d_{\max}$.

$$d_{\max} < \|\mathbf{p}_k^{\text{ref}} - \mathbf{p}_k\|_2 \tag{3.37}$$

This criterion prevents the system from exploring states that are excessively far from the desired trajectory, which are unlikely to provide useful learning signals and may lead to unstable gradient updates. If the termination condition is triggered during an episode, the quadrotor state is reset to the reference trajectory state and the loss over that horizon is not taken into account.

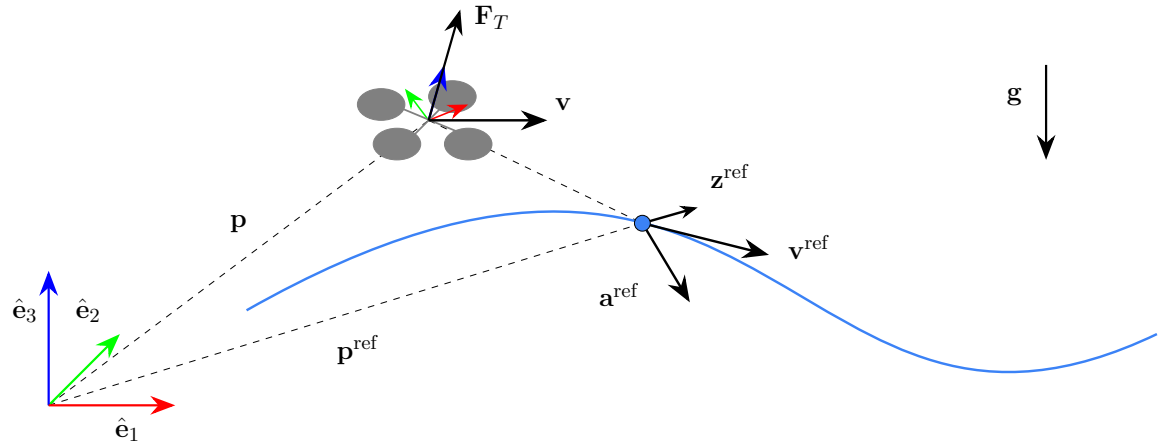


Figure 3.4: Trajectory tracking task.

Regardless of the training task, the policy network is provided with the full state of the quadrotor. For the trajectory tracking task, the observation is augmented with reference

trajectory information. Specifically, instead of providing absolute position and velocity, the observation includes tracking errors together with feedforward terms from the reference trajectory. This improves learning efficiency by explicitly encoding the control objective. The observation vector is defined as:

$$\mathbf{o}_k = \begin{bmatrix} \mathbf{p}_k^{\text{ref}} - \mathbf{p}_k & \mathbf{v}_k^{\text{ref}} - \mathbf{v}_k & \mathbf{a}_k^{\text{ref}} & \mathbf{q}_k & \boldsymbol{\omega}_k \end{bmatrix}^\top \in \mathbb{R}^{16} \tag{3.38}$$

The policies are first trained to track an arbitrary trajectory created by summing random harmonics as described in Equation 3.39, where $n = 3$, $\phi = 2.0$ and ω is initially set to one, but a curriculum is added in such a manner that the speed of the trajecotry is incremented by a certain δ when the Root Mean Square Error (RMSE) from the current and reference position is lower than a given threshold.

$$\mathbf{p}_k^{\text{ref}} = \sum_{i=1}^{n} A_i \sin\left(\omega_i k + \phi_i\right) \tag{3.39}$$

Then, the reference velocity and acceleartion are approximated as the discrete derivatives of the position, where $T_s = 0.01$.

$$\mathbf{v}_k^{\text{ref}} \approx \frac{\mathbf{p}_{k+1}^{\text{ref}} - \mathbf{p}_k^{\text{ref}}}{T_s} \tag{3.40}$$

$$\mathbf{a}_k^{\text{ref}} \approx \frac{\mathbf{v}_{k+1}^{\text{ref}} - \mathbf{v}_k^{\text{ref}}}{T_s} \tag{3.41}$$

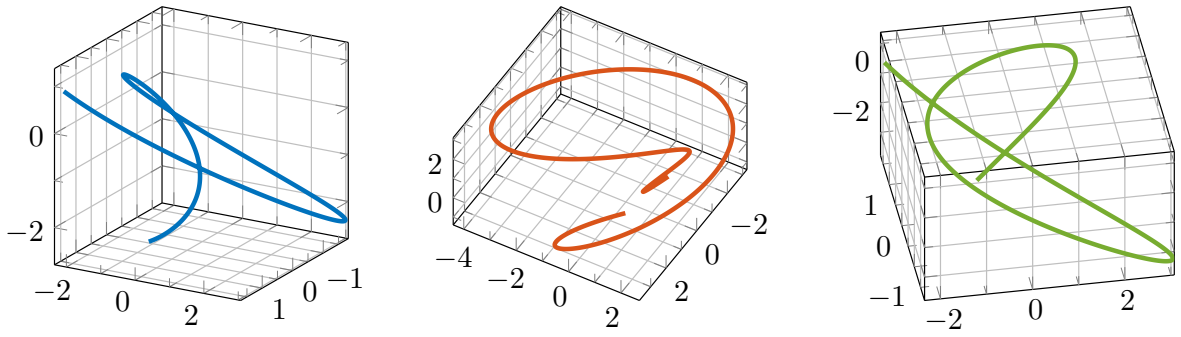


Figure 3.5: Training trajectories generated from the sum of harmonics.

3.5 Summary

To summirize the methodology, the training workflow is the following. First the user selects the training task, refered in the code as environemnt env=[tt, pp], which are either; trajectory tracking or path progress. The user also selects the control abstraction/mode cm=[srt, ctbr, lvhr, lvhr+g] which defines the control mapping function $g_{\text{cm}}()$ from the policy outputs $\boldsymbol{\pi}_{\boldsymbol{\theta}}(\mathbf{o})$ to the control inputs $\mathbf{u}$. Finally, the user introduces the number of parallel environments B, the number of episodes N, the number of steps K, the optimization horizon H and other hyperparameters such as the learning rate α, randomization percentage p, termination distance threshold $d_{\max}$, etc.

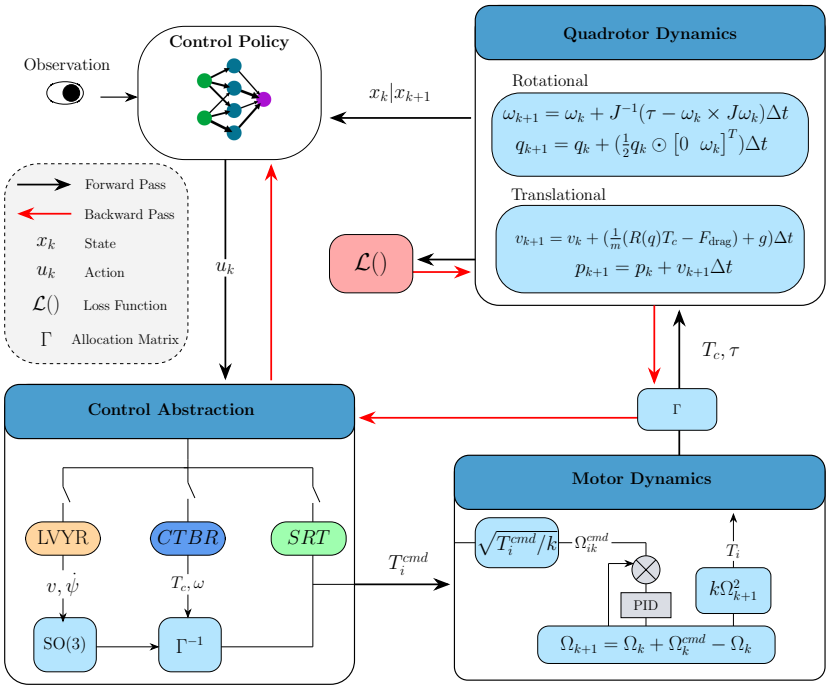


Figure 3.6: Schematics of the learning environment. The black arrows show the forward pass from the policy outputs to the loss function and the red arrows indicate the backward pass. Not thatduring the backward pass the motor dynamics are skipped.

TODO : Correct the diagram!!

Once all these parameters are set, training proceeds as follows:

Algorithm 2 Training procedure for differentiable policy optimization.

Require: Training task env $\in \{\text{tt},\ \text{pp}\}$, control mode cm $\in \{\text{srt},\ \text{ctbr},\ \text{lvhr},\ \text{lvhr+g}\}$, batch size B, episodes N, steps K, horizon H, learning rate α, simulation step $\Delta t = 0.01\,\text{s}$, other hyperparams

Initialize policy π_θ, controller g_{cm}, optimizer
for $n = 1, \ldots, N$ **do**
  Sample domain parameters $\boldsymbol{\xi}^{(i)} \sim p(\boldsymbol{\xi})$ for $i = 1, \ldots, B$ ▷ Domain randomization
  Sample reference $\mathbf{x}^{\text{ref}}_{0:K} \leftarrow \texttt{getReference}(\text{env})$ ▷ Task-dependent reference
  $\mathbf{x}_0 \leftarrow \mathbf{x}^{\text{ref}}_0, \quad J \leftarrow 0$
  for $k = 0, \ldots, K-1$ **do**
    $\mathbf{o}_k \leftarrow \texttt{getObservation}(\text{env},\ \mathbf{x}_k,\ \mathbf{x}^{\text{ref}}_k)$ ▷ Task-dependent observation
    $\mathbf{u}_k \leftarrow g_{\text{cm}}\big(\pi_\theta(\mathbf{o}_k)\big)$ ▷ Policy forward pass + control mapping
    $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k + \Delta t \cdot \mathbf{f}(\mathbf{x}_k,\ \mathbf{u}_k,\ \boldsymbol{\xi})$ ▷ Euler integration through dynamics
    $\text{term}_k \leftarrow \texttt{isTerminated}(\text{env},\ \mathbf{x}_{k+1},\ \mathbf{x}^{\text{ref}}_{k+1})$ ▷ Task-dependent termination
    $J \mathrel{+}= \dfrac{1}{B} \displaystyle\sum_{i=1}^{B} \mathcal{L}_{\text{env}}\big(\mathbf{x}^{\text{ref}}_{k+1,i},\ \mathbf{x}_{k+1,i}\big) \cdot \mathbf{1}[\neg\, \text{term}_{k,i}]$
    Reset terminated environments: $\mathbf{x}_{k+1,i} \leftarrow \mathbf{x}^{\text{ref}}_{k+1,i}$ for all i where $\text{term}_{k,i}$
    if $(k+1) \bmod H = 0$ **then** ▷ End of truncation window
      $\nabla_\theta J \leftarrow \texttt{backward}(J/H)$ ▷ BPTT through the computational graph
      $\theta \leftarrow \texttt{Adam.step}(\theta,\ \nabla_\theta J,\ \alpha)$ ▷ Parameter update
      $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_{k+1}.\texttt{detach}()$ ▷ Detach state to truncate graph
      $J \leftarrow 0$
    end if
  end for
end for
return θ

Chapter 4

Results

In this chapter, the results of the proposed differentiable simulator are presented and evaluated. First, the assessment metrics used throughout the chapter are defined in Sec. 4.1. TODO validation of the simulator Subsequently, the performance of the proposed method is compared against standard model-free RL training and a classical fixed-gain geometric controller in Sec. 4.5. Qualitative results, including trajectory visualizations, velocity profiles, and the transferability of the trained policiesare are presented. An ablation study is then conducted in Sec. 4.2 to evaluate the effect of key design choices. Finally, preliminary real-world validation results are presented in Sec. 4.4.

TODO Update this introduction, since I changed the section names....

4.1 Evaluation Criteria

This section details the metrics and experimental framework used to evaluate the performance of the proposed differentiable physics approach against a model-free PPO. The evaluation is structured around three primary axes: training efficiency, performance in terms of lap time and success rate, and sim-to-sim transferability.

4.1.1 Sample Efficiency

One of the central claims of this thesis is that leveraging analytical gradients from differentiable simulation significantly improves sample efficiency compared to standard model-free RL algorithms. To evaluate this claim, the training convergence of the proposed differentiable physics approach is compared against PPO trained on the same task, observation space, network architecture, and control abstraction. Sample efficiency is measured as the wall-clock training time and the number of environment steps required for each method to converge to a stable policy. Convergence is defined as the point at which the cumulative loss/reward ceases to improve by more than a threshold. For a fair comparison, the randomized processes use a common seed during the training process.

4.1.2 Perfromance

Lap Time and Succes Rate

Ultimately, the primary objective of ADR is to traverse a sequence of gates as fast as possible. The two metrics used to evaluate this objective are the lap time T_f and the Success Rate (SR).

The lap time T_f is defined as the time elapsed from the start of the episode until the drone passes the final gate. The success rate is defined as the ratio of successfully passed gates G_p to the total number of gates G in the racing track,

$$SR = \frac{G_p}{G} \in [0, 1]. \tag{4.1}$$

A gate g is characterized by its position $\mathbf{p}_g \in \mathbb{R}^3$, orientation $\mathbf{R}_g \in SO(3)$, and dimensions (t, w, h), corresponding to the thickness, width, and height of the gate opening, respectively. The gate frame axes are defined as,

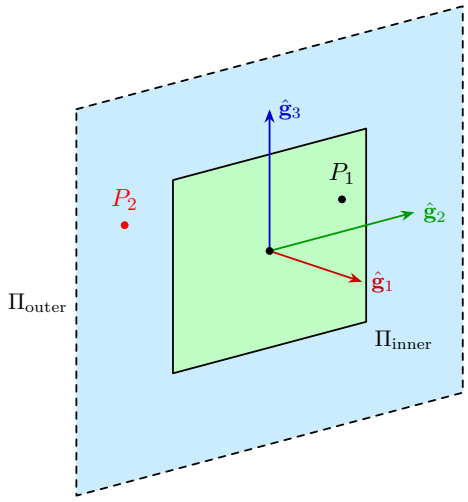


Figure 4.1: Gate traversal geometry. The success region (green) is defined with respect to the gate position $\mathbf{p}_g$, orientation $\mathbf{R}_g$ and dimensions (t, w, h). TODO add gate dimension to the plot, t,w,h

$$\hat{\mathbf{g}}_1 = \mathbf{R}_g \hat{\mathbf{e}}_1, \qquad \hat{\mathbf{g}}_2 = \mathbf{R}_g \hat{\mathbf{e}}_2, \qquad \hat{\mathbf{g}}_3 = \mathbf{R}_g \hat{\mathbf{e}}_3, \tag{4.2}$$

where $\hat{\mathbf{g}}_1$, $\hat{\mathbf{g}}_2$, and $\hat{\mathbf{g}}_3$ denote the forward (normal), lateral , and vertical axes of the gate, respectively. The position of the drone relative to the gate, expressed in the gate frame, is,

$$\mathbf{p}_k^g = \mathbf{R}_g^\top (\mathbf{p}_k - \mathbf{p}_g) = \begin{bmatrix} p_1^g & p_2^g & p_3^g \end{bmatrix}^\top. \tag{4.3}$$

A gate g is considered successfully passed at timestep k if,

$$|p_1^g| < \frac{t}{2} \quad \wedge \quad |p_2^g| \leq \frac{w}{2} \quad \wedge \quad |p_3^g| \leq \frac{h}{2}, \tag{4.4}$$

and a gate is unsuccessfully passed when,

$$|p_1^g| < \frac{t}{2} \quad \wedge \quad \left(|p_2^g| > \frac{w}{2} \ \vee \ |p_3^g| > \frac{h}{2} \right). \tag{4.5}$$

Root Mean Square Error

When a referene path is used, another performance metric utilized is to compute the RMSE between the drone's position and the reference path.

4.1.3 Transferability and Robustness

Learned policies are prone to overfitting to the training environment and may fail to generalize when deployed under conditions that deviate from those seen during training, due to model mismatches and unmodeled effects. To mitigate this, DR is employed during training as described in section 3.3.

To assess transferability, the policies trained in the proposed differentiable simulator are evaluated in a high-fidelity simulation environment using the MRS UAV stack in Gazebo [29]. Since the differentiable physics and model-free RL policies are optimized under different paradigms, the sim-to-sim performance gap is analyzed for each method, measuring the degradation in success rate SR and lap time T_f relative to the results obtained in the training simulator. Additionally, robustness to external disturbances is evaluated by exposing the trained policies to wind perturbations during deployment, without any retraining or fine-tuning.

4.2 Control Abstraction Evaluation

In this section, we evaluate how the implemented control abstractions affect the sample efficiency, racing performance, as well as the transferability and robustness of the learned control policies.

4.2.1 Sample Efficency

To evaluate sample efficiency, the four policies are trained for 40 million steps under identical conditions, including the same randomization settings, loss function, network architecture, learning rate, and hyperparameters. We then analyze how the loss evolves throughout training.

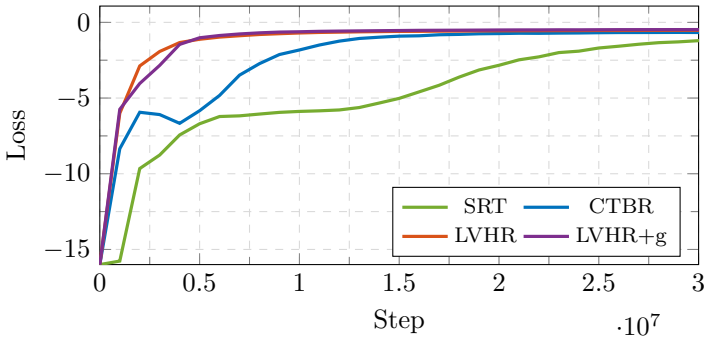


Figure 4.2: Loss verus time step for the different control abstractions.

From Figure 4.2, it can be observed that, as discussed in section 3.2, the SRT policy requires the longest training time to converge, since the network must implicitly learn the mapping between motor commands and body rates. Although this approach is slower to train, it has the advantage of requiring no controller gains or manual tuning. In contrast, the CTBR policy converges significantly faster, reaching convergence after approximately 1.5 million steps. The geometric control abstractions, LVHR and LVHR+g, converge even faster, requiring only around 0.5 million steps. Despite their improved convergence speed, its woth noting that latter policies rely on carefully tuned gains to ensure stable behavior, which can be time consuming and task-dependant.

4.2.2 Performance

To evaluate performance, the policies are further trained on a specific racing track. The selected track is commonly used in the literature, as it contains straight sections, high-curvature segments, and a split-S gate configuration, making it suitable for benchmarking agile flight performance. This circuit is commonly referred to as the UZH track, as it was designed by the University of Zurich.

To generate trajectories passing through the gates, the time-optimal gate traversal planner proposed in [42] is used. The planner generates trajectories that account for both the gate geometry and the quadrotor dynamics. Training directly on the optimal trajectory consistently resulted in failure to track the reference. We hypothesize that the optimization becomes trapped in local minima, preventing the policy from converging to a stable solution. To facilitate learning, a curriculum-based training strategy is adopted. Initially, the trajectory speed is scaled down to 10% of the original reference speed. Once an episode achieves an RMSE below 0.15 m, the trajectory speed is increased by an additional 10%. Using this approach, the policies are able to track the trajectory at up to 60% of the original speed, which remains significantly below the optimal performance. This limitation requires further investigation, particularly regarding the design of the loss function and the neural network architecture.

Table 4.1 and Figure 4.3 present the tracking results of the four control abstractions on the UZH track at 60% of the reference speed, where the policies achieve a maximum speed of 10 m/s.

Table 4.1: Position and Velocity RMSE of the control abstractions, mean ± std over 5 runs.

RMSE	SRT	CTBR	LVHR	LVHR+g
Position [m]	0.225 ± 0.031	0.334 ± 0.008	$\mathbf{0.132 \pm 0.003}$	0.201 ± 0.003
Velocity [m/s]	0.190 ± 0.002	$\mathbf{0.180 \pm 0.002}$	0.219 ± 0.027	0.224 ± 0.008

The results presented in Figure 4.3 and Table 4.1 indicate that the LVHR control abstraction achieves the lowest position tracking error, while CTBR obtains the lowest velocity tracking error. At this suboptimal speed, all policies achieve a SR of 100%, with no gate collisions occurring during the experiments.

Furthermore, the policies exhibit broadly similar behavior. The collective thrust commands follow comparable patterns across all control abstractions. The SRT and LVHR+g policies appear to lag slightly behind the reference trajectory, whereas the CTBR policy tends to move ahead of the reference and produces the most oscillatory control commands. Although the LVHR policy achieves the best position tracking performance overall, its velocity tracking error increases significantly toward the end of the trajectory.

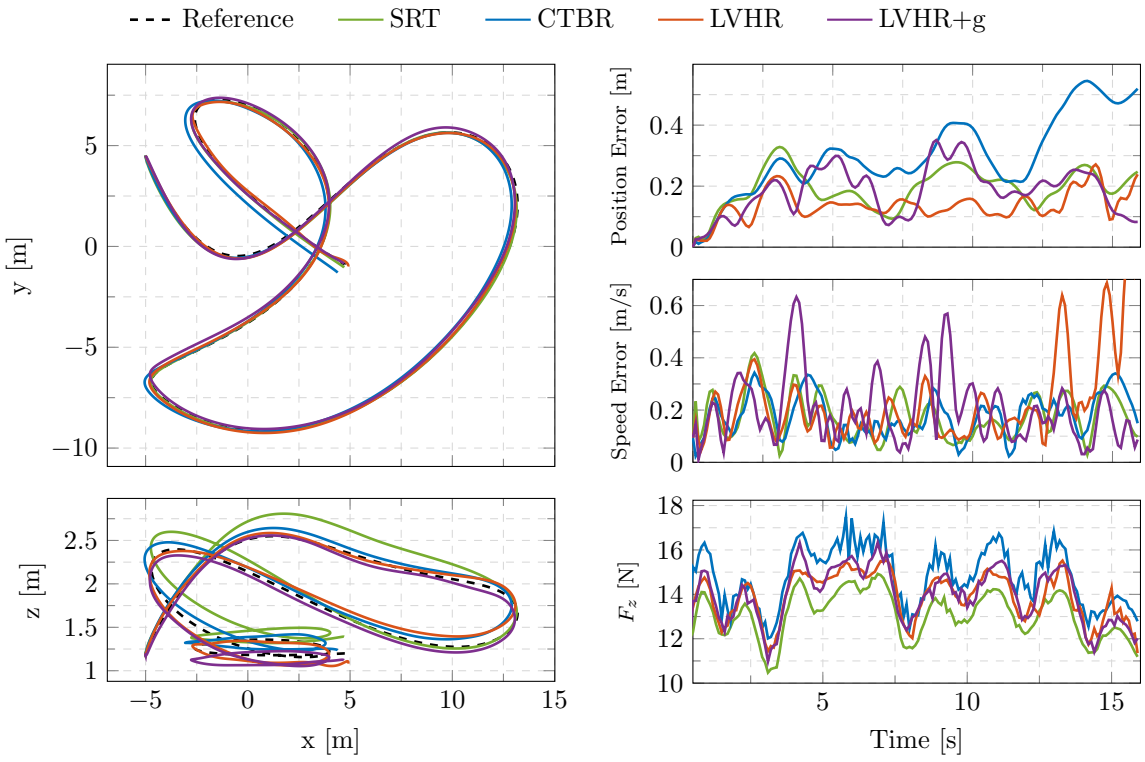


Figure 4.3: Tracking error of the different control abstractions for the UZH racing track.

Analyzing the commands of each control abstraction individually reveals information in policy behavior. For the SRT policy, in fig:srt-action it can be seen that the rotor commands follow a trend similar to the collective thrust profile shown in Figure 4.3. The policy successfully learns to rotate the quadrotor by generating differential thrust between the rotors. Around 12 s, high-frequency oscillations can be observed in the commands; this interval corresponds to the split-S section of the track, where aggressive rotational maneuvers are required.

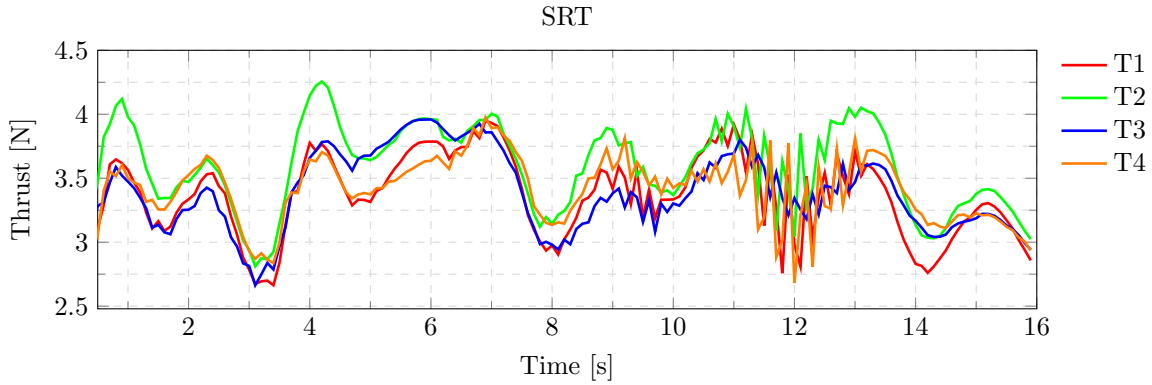


Figure 4.4: Commanded single rotor thrusts for the UZH trajectory.

From Figure 4.5, it can be observed that the control commands are relatively jittery. The pitch and roll moments are the most aggressive, reaching peak values of approximately 0.05 N · m during the split-S maneuver. In contrast, the yaw moments remain close to 0.0 N · m, since maintaining the heading aligned with the gates was not explicitly prioritized during training.

The platform has a maximum TWR of 6.9, corresponding to a maximum thrust of approximately 82 N. For the evaluated trajectory, the maximum commanded thrust is around 14 N, which corresponds to only 17% of the maximum available thrust.

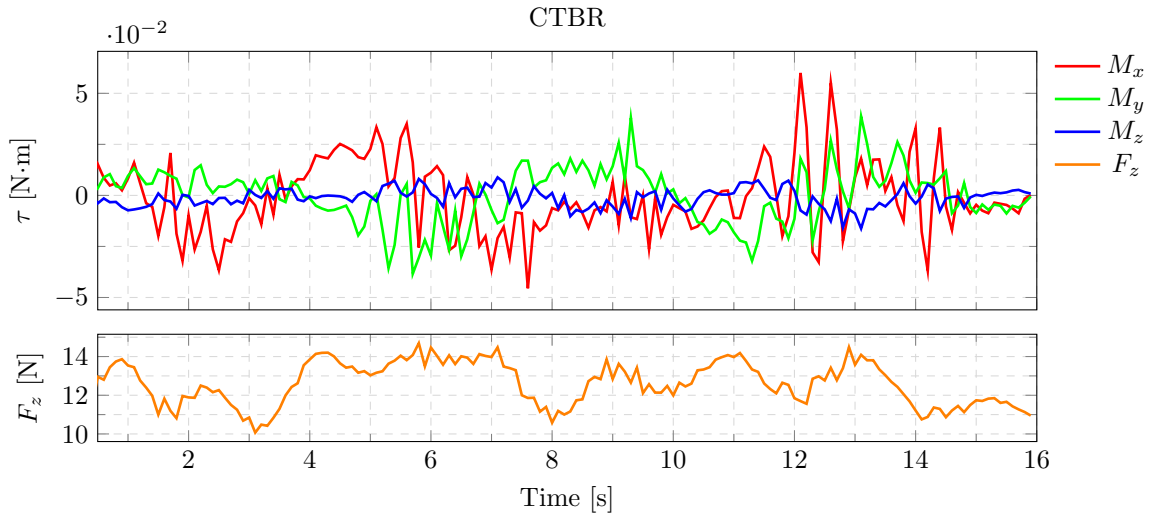


Figure 4.5: Commanded collective thrust and body rates for the UZH trajectory.

The velocity and heading-rate commands shown in Figure 4.6 exhibit a similar structure throughout the trajectory. The commanded velocities along the body x and body y axes have the largest magnitudes, while the heading rate and vertical velocity v_z remain close to zero for most of the flight.

The main advantage of this control abstraction is the smoothness of the generated commands. Additionally, as shown in Figure 4.2, this policy achieves the fastest convergence during training.

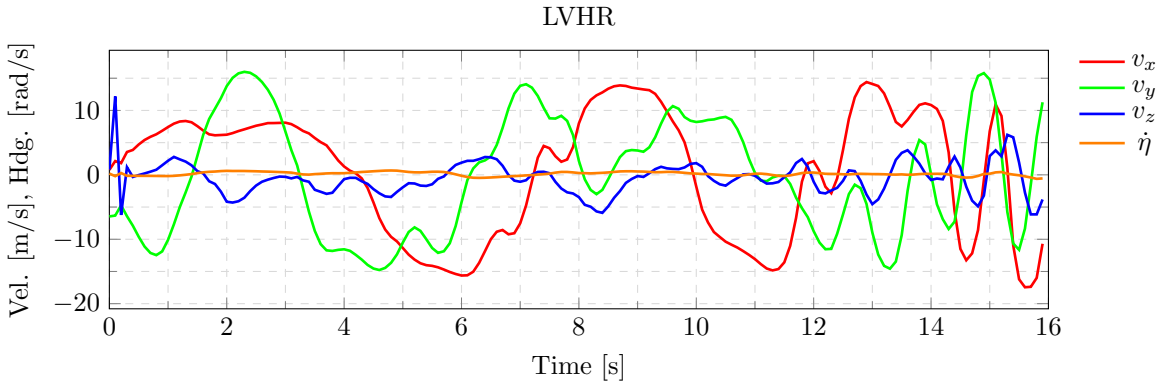


Figure 4.6: Commanded linear velocities and heading rate for the UZH trajectory.

Finally, in the current setup, augmenting the policy to output the gains of the geometric controller does not appear to provide significant improvements. The gains are initialized with the same values used in the LVHR policy, namely $k_v = 0.6$, $k_R = 0.2$, and $k_\omega = 0.05$. However, in this case, the policy outputs scaling factors between 0 and 1 that modulate these gains.

As shown in Figure 4.7, the gains remain relatively constant throughout the trajectory, raising the question of whether the feature justified in the current setup. Nevertheless, it is difficult to draw definitive conclusions from this experiment, since the evaluated trajectory remains far from optimal and the actuators operate well below saturation for most of the flight.

Further investigation will be conducted in future work, as we believe this approach could be beneficial in more aggressive flight regimes or under varying operating conditions, potentially providing advantages over fixed-gain geometric controllers. As discussed in [43], "it is widely known that this controller design approach provides better stability and performance results for slowly varying parameters" [44].

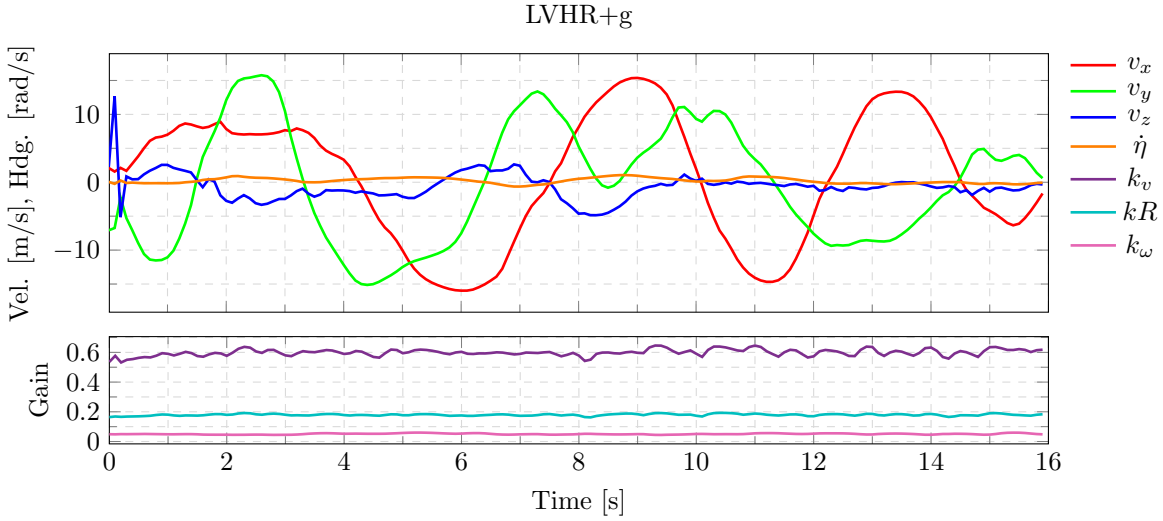


Figure 4.7: Commanded linear velocities, heading rate and controller gains for the UZH trajectory.

4.2.3 Transferability

To evaluate how the controllers transfer to a high-fidelity simulator, we implemented a neural nework controller as a ROS node in C++ and integrated into the MRS UAV system [29] within Gazebo. The CTBR policy was selected for evaluation, as this control abstraction is widely used in aerial robotics and is currently the only one supported by the MRS UAV Pixhawk 4 [45] Application Programming Interface (API). In future work, the remaining control abstractions will also be implemented within the PX4 interface, particularly the LVHR+g approach. Nevertheless, the CTBR controller provides a suitable baseline for evaluating transferability.

Using this setup, we compare the position and velocity RMSE obtained in our simulator against the results in Gazebo. In Gazebo, two configurations are considered: one using ground-truth state estimation and another using a more realistic state estimator representative of real flight conditions.

4.3 Surrogate Gradient Analysis

In this section, we analyze the effect of using surrogate gradients during the policy optimization process. The gradient is backpropagated from the loss function to the policy parameters through the chain rule. As discussed in section 2.4, the gradients are computed numerically from the computational graph. Certain operations, such as Equation 3.10, can be highly sensitive to small perturbations, leading to unstable gradients and making the optimization process more difficult.

This behavior can be observed in Figure 4.8, where the loss and gradient norm are plotted against the training steps for two different approaches: a simplified backpropagated model that omits motor dynamics, and the full model including the motor dynamics. At the beginning of training, both approaches exhibit similar behavior. However, the full model eventually converges to a suboptimal solution and fails to track the trajectory successfully. By analyzing the gradient norm of the full model, larger oscillations can be observed, which we believe are responsible for the unstable optimization and suboptimal convergence.

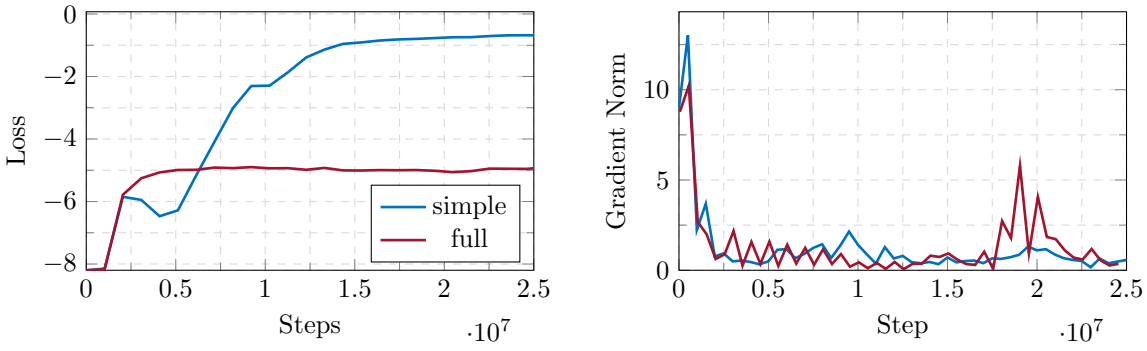


Figure 4.8: Gradients.

4.3.1 Robustness

4.4 Real World Validation

On the 16th of April 2026, during the MRS testing campaign, we conducted a real-world flight experiment. At that stage, the simulator was still in an early phase of development; actuator dynamics and aerodynamic drag were not yet modeled, and policy training relied on ground-truth state information.

As a result, there was a significant mismatch between the simulator and real flight conditions. This led to poor trajectory tracking performance, with position and velocity RMSE values of 6.11 m and 4.25 m/s, respectively. Although the overall performance was limited, Figure 4.9 shows that the general shape of the trajectory was still preserved. This suggests that DR contributed positively to policy transferability in real-world conditions. Additionally, the experiment provided valuable insights and data that were later used to improve the simulator.

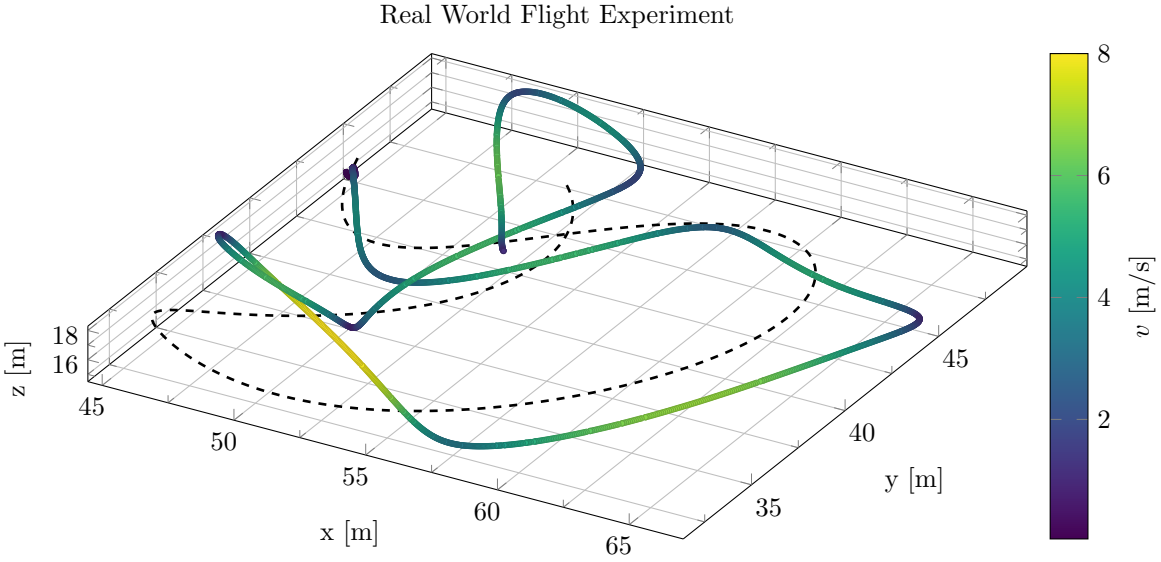


Figure 4.9: Real world flight data, UZH track trajectory tracking.

4.5 Model-Free and Model-Based evaluation

In this section, we compare two policies, one optimized using model-free RL with PPO, and the other using model-based RL with BPTT. The comparison is performed in terms of sample efficiency, mean reward, performance, transferability, and robustness.

4.5.1 Training Time

To evaluate the training time we measure the wall time agians the number of steps. It can be observed that the relationship between the steps and time is linear. Taking for BPTT **7x** times less clock time to execute the same number of time steps than PPO.

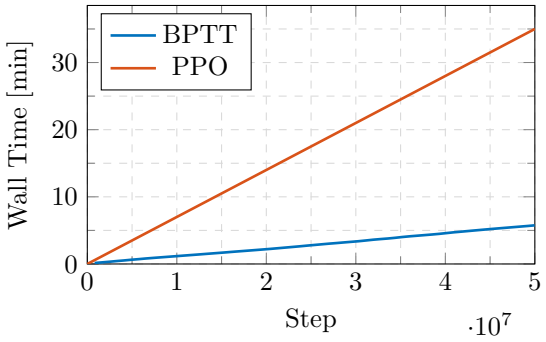


Figure 4.10: Wall time versus training steps for PPO and BPTT.

It is worth mentioning that the wall time is highly dependant on the frequency at which the network is optimized, as oftelny this procedure is the most time consuming. For this comparison, the policies are optimized when losses for 50 stpes for 1024 parallel environments are collected, that is to say, every 51200 stpes.

4.5.2 Discrete and Continuous Rewards

TODO

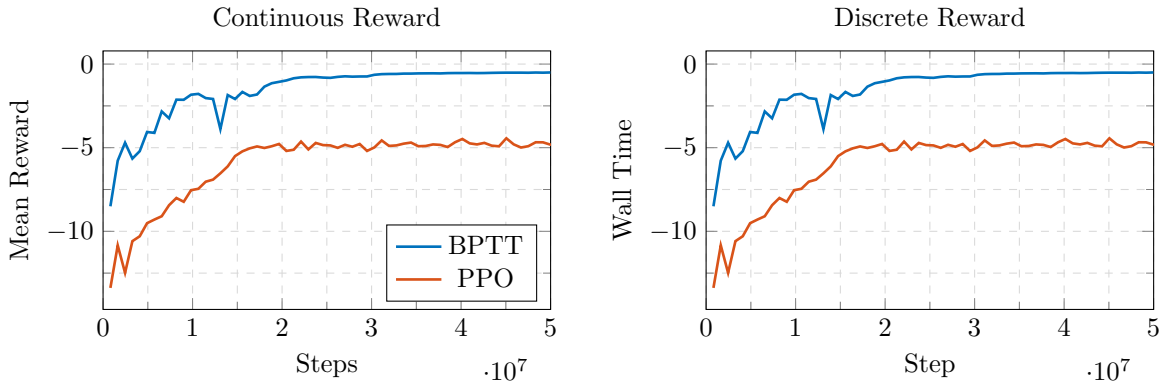


Figure 4.11: Comparison between BPTT and PPO when the loss is countinuous or discrete.

4.5.3 Performance

$$\mathbf{p}(\theta) = (-\text{scale} \cdot \sin(\theta)\cos(\theta),\ -\text{scale} \cdot \cos(\theta),\ \text{h}) \tag{4.6}$$

where $\theta = \omega \cdot t \cdot dt$ and $\omega = \text{speed}/\text{scale}$

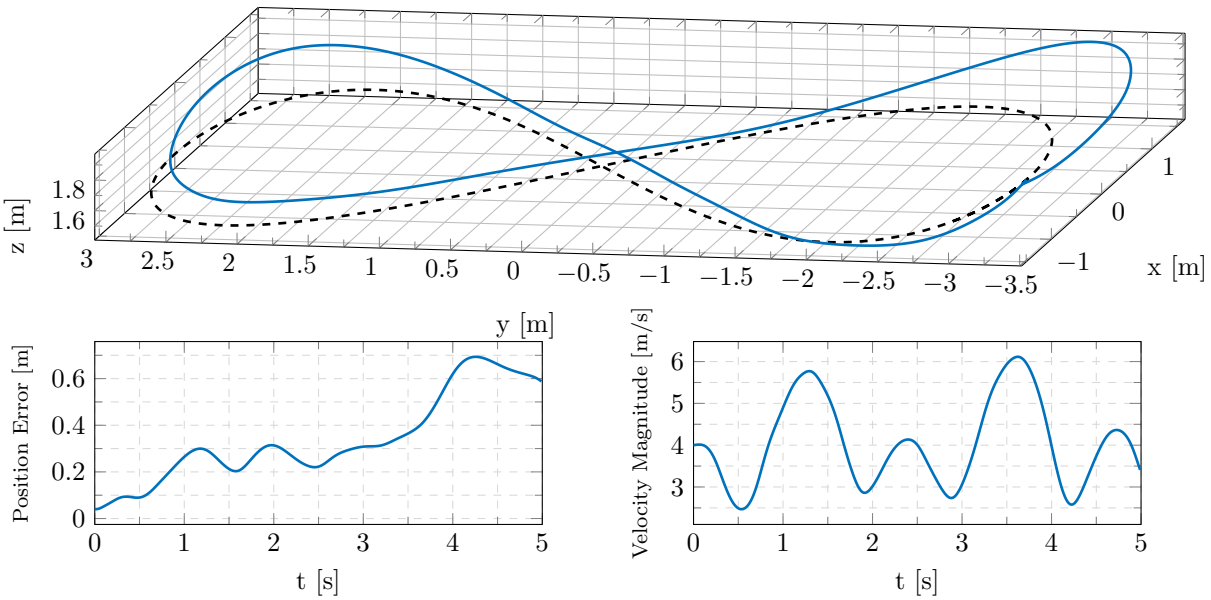


Figure 4.12: TODO

4.5.4 Transferability and Robustness

Chapter 5

Conclusion

Chapter 6

Future Directions

Chapter 7

References

[1] Drew Hanover et al. "Autonomous Drone Racing: A Survey". en. In: *IEEE Transactions on Robotics* 40 (2024). arXiv:2301.01755 [cs], pp. 3044–3067. ISSN: 1552-3098, 1941-0468. DOI: 10.1109/TRO.2024.3400838. URL: http://arxiv.org/abs/2301.01755 (visited on 02/13/2026).

[2] Daniel Hert et al. "MRS Drone: A Modular Platform for Real-World Deployment of Aerial Multi-Robot Systems". en. In: *Journal of Intelligent & Robotic Systems* 108.4 (July 2023), p. 64. ISSN: 1573-0409. DOI: 10.1007/s10846-023-01879-2. URL: https://doi.org/10.1007/s10846-023-01879-2 (visited on 06/03/2026).

[3] Philipp Foehn et al. "Agilicious: Open-source and open-hardware agile quadrotor for vision-based flight". en. In: *Science Robotics* 7.67 (June 2022), eabl6259. ISSN: 2470-9476. DOI: 10.1126/scirobotics.abl6259. URL: https://www.science.org/doi/10.1126/scirobotics.abl6259 (visited on 05/12/2026).

[4] Rudolph Emil Kalman. "A New Approach to Linear Filtering and Prediction Problems". In: *Transactions of the ASME–Journal of Basic Engineering* 82.Series D (1960), pp. 35–45.

[5] E.A. Wan and R. Van Der Merwe. "The unscented Kalman filter for nonlinear estimation". en. In: *Proceedings of the IEEE 2000 Adaptive Systems for Signal Processing, Communications, and Control Symposium (Cat. No.00EX373)*. Lake Louise, Alta., Canada: IEEE, 2000, pp. 153–158. ISBN: 978-0-7803-5800-3. DOI: 10.1109/ASSPCC.2000.882463. URL: http://ieeexplore.ieee.org/document/882463/ (visited on 06/03/2026).

[6] Adam Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. arXiv:1912.01703 [cs]. Dec. 2019. DOI: 10.48550/arXiv.1912.01703. URL: http://arxiv.org/abs/1912.01703 (visited on 05/07/2026).

[7] James Bradbury et al. *JAX: composable transformations of Python+NumPy programs*. Version 0.3.13. 2018. URL: http://github.com/jax-ml/jax.

[8] Martín Abadi et al. *TensorFlow: A system for large-scale machine learning*. arXiv:1605.08695 [cs]. May 2016. DOI: 10.48550/arXiv.1605.08695. URL: http://arxiv.org/abs/1605.08695 (visited on 05/07/2026).

[9] Yuang Zhang et al. "Back to Newton's Laws: Learning Vision-based Agile Flight via Differentiable Physics". en. In: *Nature Machine Intelligence* 7.6 (June 2025). arXiv:2407.10648 [cs], pp. 954–966. ISSN: 2522-5839. DOI: 10.1038/s42256-025-01048-0. URL: http://arxiv.org/abs/2407.10648 (visited on 02/23/2026).

[10] Hyungpil Moon et al. "The IROS 2016 Competitions [Competitions]". In: *IEEE Robotics & amp; Automation Magazine* 24.1 (Mar. 2017), pp. 20–29. ISSN: 1558-223X. DOI: 10.1109/mra.2016.2646090. URL: http://dx.doi.org/10.1109/MRA.2016.2646090.

[11] Sunggoo Jung et al. https://sunyouh.github.io/projects/adr2017. [Accessed 30-04-2026]. 2017.

[12] *Competitions — iros — iros2018.org*. https://www.iros2018.org/competitions. [Accessed 30-04-2026].

[13] *AlphaPilot AI Drone Innovation Challenge — lockheedmartin.com*. https://www.lockheedmartin.com/en-us/news/events/ai-innovation-challenge.html. [Accessed 30-04-2026].

[14] Philipp Foehn et al. "AlphaPilot: Autonomous Drone Racing". en. In: (Aug. 2021). arXiv:2005.12813 [cs]. DOI: 10.48550/arXiv.2005.12813. URL: http://arxiv.org/abs/2005.12813.

[15] Elia Kaufmann et al. "Champion-level drone racing using deep reinforcement learning". en. In: *Nature* 620.7976 (Aug. 2023), pp. 982–987. ISSN: 0028-0836, 1476-4687. DOI: 10.1038/s41586-023-06419-4. URL: https://www.nature.com/articles/s41586-023-06419-4.

[16] John Schulman et al. *Proximal Policy Optimization Algorithms*. 2017. DOI: 10.48550/ARXIV.1707.06347. URL: https://arxiv.org/abs/1707.06347.

[17] Yunlong Song et al. "Reaching the Limit in Autonomous Racing: Optimal Control versus Reinforcement Learning". en. In: *Science Robotics* 8.82 (Sept. 2023). arXiv:2310.10943 [cs], eadg1462. ISSN: 2470-9476. DOI: 10.1126/scirobotics.adg1462. URL: http://arxiv.org/abs/2310.10943 (visited on 02/13/2026).

[18] Angel Romero et al. *Dream to Fly: Model-Based Reinforcement Learning for Vision-Based Drone Flight*. en. arXiv:2501.14377 [cs]. Jan. 2025. DOI: 10.48550/arXiv.2501.14377. URL: http://arxiv.org/abs/2501.14377 (visited on 02/13/2026).

[19] Aderik Verraest et al. *SkyDreamer: Interpretable End-to-End Vision-Based Drone Racing with Model-Based Reinforcement Learning*. en. arXiv:2510.14783 [cs]. Oct. 2025. DOI: 10.48550/arXiv.2510.14783. URL: http://arxiv.org/abs/2510.14783 (visited on 02/13/2026).

[20] Danijar Hafner et al. *Mastering Diverse Domains through World Models*. en. arXiv:2301.04104 [cs]. Apr. 2024. DOI: 10.48550/arXiv.2301.04104. URL: http://arxiv.org/abs/2301.04104 (visited on 02/13/2026).

[21] Xinhong Zhang et al. *DiffAero: A GPU-Accelerated Differentiable Simulation Framework for Efficient Quadrotor Policy Learning*. en. arXiv:2509.10247 [cs]. Sept. 2025. DOI: 10.48550/arXiv.2509.10247. URL: http://arxiv.org/abs/2509.10247 (visited on 02/13/2026).

[22] Johannes Heeg, Yunlong Song, and Davide Scaramuzza. "Learning Quadrotor Control From Visual Features Using Differentiable Simulation". en. In: *2025 IEEE International Conference on Robotics and Automation (ICRA)*. arXiv:2410.15979 [cs]. May 2025, pp. 4033–4039. DOI: 10.1109/ICRA55743.2025.11128641. URL: http://arxiv.org/abs/2410.15979 (visited on 02/13/2026).

[23] Jiahe Pan et al. *Learning on the Fly: Rapid Policy Adaptation via Differentiable Simulation*. en. arXiv:2508.21065 [cs]. Jan. 2026. DOI: 10.48550/arXiv.2508.21065. URL: http://arxiv.org/abs/2508.21065 (visited on 02/13/2026).

[24] Yunfan Ren et al. *Learning Agile Quadrotor Flight in the Real World*. arXiv:2602.10111 [cs]. Feb. 2026. DOI: 10.48550/arXiv.2602.10111. URL: http://arxiv.org/abs/2602.10111 (visited on 02/16/2026).

[25] Robin Ferede et al. "One Net to Rule Them All: Domain Randomization in Quadcopter Racing Across Different Platforms". In: (2025). DOI: 10.48550/ARXIV.2504.21586. URL: https://arxiv.org/abs/2504.21586.

[26] Antonio Loquercio, Alessandro Saviolo, and Davide Scaramuzza. "AutoTune: Controller Tuning for High-Speed Flight". en. In: *IEEE Robotics and Automation Letters* 7.2 (Apr. 2022), pp. 4432–4439. ISSN: 2377-3766, 2377-3774. DOI: 10.1109/LRA.2022.3146897. URL: https://ieeexplore.ieee.org/document/9696365/ (visited on 05/12/2026).

[27] Nicholas Metropolis et al. "Equation of State Calculations by Fast Computing Machines". en. In: *The Journal of Chemical Physics* 21.6 (June 1953), pp. 1087–1092. ISSN: 0021-9606, 1089-7690. DOI: 10.1063/1.1699114. URL: https://pubs.aip.org/jcp/article/21/6/1087/202680/Equation-of-State-Calculations-by-Fast-Computing (visited on 06/08/2026).

[28] Komal Khuwaja et al. "PID Controller Tuning Optimization with Genetic Algorithms for a Quadcopter". en. In: *Recent Innovations in Mechatronics* 5.1 (Apr. 2018), pp. 1–7. ISSN: 2064-9622. DOI: 10.17667/riim.2018.1/11. URL: https://ojs.lib.unideb.hu/rIim/article/view/3840 (visited on 06/08/2026).

[29] Tomas Baca et al. "The MRS UAV System: Pushing the Frontiers of Reproducible Research, Real-world Deployment, and Education with Autonomous Unmanned Aerial Vehicles". en. In: *Journal of Intelligent & Robotic Systems* 102.1 (May 2021), p. 26. ISSN: 0921-0296, 1573-0409. DOI: 10.1007/s10846-021-01383-5. URL: https://link.springer.com/10.1007/s10846-021-01383-5 (visited on 05/11/2026).

[30] H.J. Sussmann and J.C. Willems. "300 years of optimal control: from the brachystochrone to the maximum principle". In: *IEEE Control Systems Magazine* 17.3 (June 1997), pp. 32–44. ISSN: 1941-000X. DOI: 10.1109/37.588098. URL: https://ieeexplore.ieee.org/document/588098/ (visited on 04/01/2026).

[31] Daniel Mellinger and Vijay Kumar. "Minimum snap trajectory generation and control for quadrotors". en. In: *2011 IEEE International Conference on Robotics and Automation*. Shanghai, China: IEEE, May 2011, pp. 2520–2525. ISBN: 978-1-61284-386-5. DOI: 10.1109/ICRA.2011.5980409. URL: http://ieeexplore.ieee.org/document/5980409/ (visited on 02/13/2026).

[32] Zdeněk Hurák. *Model predictive control (MPC); B(E)3M35ORR; Optimal and Robust Control — hurak.github.io*. https://hurak.github.io/orr/discr_dir_mpc.html. [Accessed 07-06-2026]. 2025.

[33] Qingyan Huang. "Model-Based or Model-Free, a Review of Approaches in Reinforcement Learning". In: *2020 International Conference on Computing and Data Science (CDS)*. Aug. 2020, pp. 219–221. DOI: 10.1109/CDS49703.2020.00051. URL: https://ieeexplore.ieee.org/document/9275964/ (visited on 05/12/2026).

[34] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Second. The MIT Press, 2018. URL: http://incompleteideas.net/book/the-book-2nd.html.

[35] Volodymyr Mnih et al. "Playing Atari with Deep Reinforcement Learning". en. In: ().

[36] John Schulman et al. *Trust Region Policy Optimization*. arXiv:1502.05477 [cs.LG]. Apr. 2017. DOI: 10.48550/arXiv.1502.05477. URL: http://arxiv.org/abs/1502.05477 (visited on 06/08/2026).

[37] Diederik P. Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. arXiv:1412.6980 [cs]. Jan. 2017. DOI: 10.48550/arXiv.1412.6980. URL: http://arxiv.org/abs/1412.6980 (visited on 05/07/2026).

[38] Taeyoung Lee, Melvin Leok, and N. Harris McClamroch. "Geometric tracking control of a quadrotor UAV on SE(3)". In: *49th IEEE Conference on Decision and Control (CDC)*. ISSN: 0191-2216. Dec. 2010, pp. 5420–5425. DOI: 10.1109/CDC.2010.5717652. URL: https://ieeexplore.ieee.org/document/5717652/ (visited on 03/09/2026).

[39] K.J Åström. "Optimal control of Markov processes with incomplete state information". In: *Journal of Mathematical Analysis and Applications* 10.1 (1965), pp. 174–205. ISSN: 0022-247X. DOI: https://doi.org/10.1016/0022-247X6590154-X. URL: https://www.sciencedirect.com/science/article/pii/0022247X6590154X.

[40] Lilian Weng. "Domain Randomization for Sim2Real Transfer". In: *lilianweng/github/io* (2019). URL: https://lilianweng.github.io/posts/2019-05-05-domain-randomization/.

[41] Dingqi Zhang et al. "A Learning-Based Quadcopter Controller With Extreme Adaptation". en. In: *IEEE Transactions on Robotics* 41 (2025), pp. 3948–3964. ISSN: 1552-3098, 1941-0468. DOI: 10.1109/TRO.2025.3577037. URL: https://ieeexplore.ieee.org/document/11025148/ (visited on 02/12/2026).

[42] Chao Qin et al. *Time-Optimal Gate-Traversing Planner for Autonomous Drone Racing*. en. arXiv:2309.06837 [cs]. May 2024. DOI: 10.48550/arXiv.2309.06837. URL: http://arxiv.org/abs/2309.06837 (visited on 02/13/2026).

[43] Heithem Boufrioua and Boubekeur Boukhezzar. "Gain Scheduling: A Short Review". In: *2022 2nd International Conference on Advanced Electrical Engineering (ICAEE)*. Oct. 2022, pp. 1–6. DOI: 10.1109/ICAEE53772.2022.9961975. URL: https://ieeexplore.ieee.org/document/9961975/ (visited on 05/18/2026).

[44] W.J. Rugh. "Analytical framework for gain scheduling". In: *IEEE Control Systems Magazine* 11.1 (Jan. 1991), pp. 79–84. ISSN: 1941-000X. DOI: 10.1109/37.103361. URL: https://ieeexplore.ieee.org/document/103361/ (visited on 05/18/2026).

[45] Multi-robot Systems (MRS) group at Czech Technical University in Prague. *MRS UAV PX4 API Plugin*. May 18, 2026. URL: https://github.com/ctu-mrs/mrs_uav_px4_api/tree/master.

Appendix A

Appendix A

A.1 Simulator Validation

As detailed in **??**, our simulator utilizes a simplified model of quadrotor dynamics and aerodynamics. To maintain a computationally efficient and generalizable framework, we omit secondary effects such as battery depletion on thrust, complex aerodynamic effects, and the propellers rotational inertia. While these unmodeled dynamics can cumulatively lead to a significant sim-to-real gap, we aim to address this mismatch through DR. To validate the reliability of this simplified approach, we benchmark our simulator's behavior against Gazebo, a high-fidelity physics engine widely adopted by the robotics community. Specifically, we utilize the UAV models provided by the MRS UAV System within the Gazebo environment as our ground-truth reference for comparison.

We separelty asses the respones of the body rates against a unit step, a variable amplitude and a variable frequency signals.

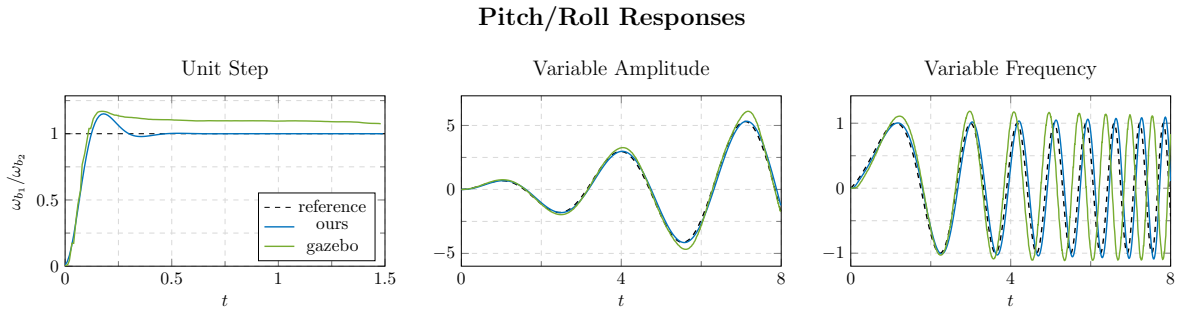


Figure A.1: Pitch/Roll responses to unit step, variable amblitude and variable frequency signals.

For the pitch and roll responses, the behavior observed in Gazebo and in our simulator is consistent. Minor discrepancies can be observed in the response of integration of the unit-step error, together with a slight overshoot in the variable-frequency signal.

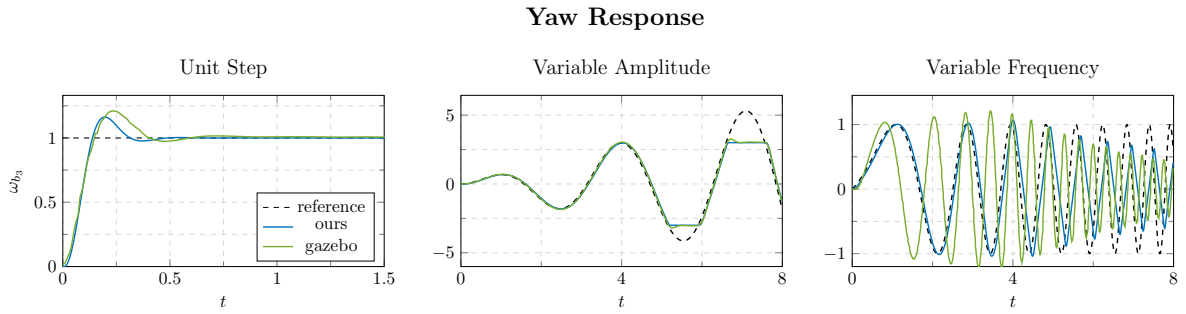


Figure A.2: Yaw response to unit step, variable amblitude and variable frequency signals.

Similarly, the yaw reponse are close but there is some mismathc on the variable freuqency signal. In our simulator we saturate the yaw rate to ± 3 rad/s and the yaw acceleartion to ± 5 rad/s^2 to match with Gazebo.